BRK (00)	Cycles: 7	* Size: 2
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**	** Store as OpCode. PushB = true.
1.1		Inc. **PC**.
1.2	Read **PC**	Discard.
2.1		Inc. **PC**.
2.2	*** Write to u8(**S**+0) | $0100	Write **PC**.H.
3.1		
3.2	*** Write to u8(**S**-1) | $0100	Write **PC**.L.
4.1		Use Interrupt flags to select Vector.
		If RESET, Vector = $FFFC.
		Else if NMI, Vector = $FFFA.
		Else if IRQ/BRK, Vector = $FFFE.
		If not BRK, PushB = false.
4.2	*** Write to u8(**S**-2) | $0100	If PushB, Write (**P** | (**X** flag)), otherwise write **P**.
5.1		Dec. **S** by 3.
5.2	Read Vector	Store as Address.L.
6.1		**I** flag = 1, **M** flag = 1, **X** flag = 0.
		Enable updates to **PC** (disabled by NMI/IRQ).
6.2	Read Vector + 1	Store as Address.H.
7.1		**PC** = Address.
7.2	Read **PC**	Store as OpCode.
+X.1		

* Concerning the BRK instruction, you should also note that although its second byte is basically a “don’t care” byte – that is, it can have any value - the BRK (and COP instruction as well) is a two-byte instruction, the second byte sometimes is used as a signature byte to determine the nature of the BRK being executed. When an RTI instruction is executed, control always returns to the second byte past the BRK opcode. — assembly-programming-manual-for-w65c816.pdf

** If NMI, IRQ, or RESET, OpCode is forced to 0 and writes to PC are disabled. This is done on ALL “Store as Opcode” half-cycles, but only noted here.

*** If handling a RESET, the writes turn into reads. The databus is populated but ignored.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	00	1	BRK	0006	f0	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	00	1	BRK	0006	f0	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	09	1		0007	f0	4c	e8	fd	nv-BdIzc	BRK	T2	98	98	98
8	0007	09	1		0007	f0	4c	e8	fd	nv-BdIzc	BRK	T2	98	98	30
9	01fd	09	0		0008	f0	4c	e8	fd	nv-BdIzc	BRK	T3	ff	fd	30
9	01fd	00	0		0008	f0	4c	e8	fd	nv-BdIzc	BRK	T3	ff	fd	fc
10	01fc	09	0		0008	f0	4c	e8	fd	nv-BdIzc	BRK	T4	ff	fc	fc
10	01fc	08	0		0008	f0	4c	e8	fd	nv-BdIzc	BRK	T4	ff	fc	fb
11	01fb	09	0		0008	f0	4c	e8	fd	nv-BdIzc	BRK	T5	ff	fb	fb
11	01fb	34	0		0008	f0	4c	e8	fd	nv-BdIzc	BRK	T5	ff	fb	fa
12	fffe	00	1		0008	f0	4c	e8	fa	nv-BdIzc	BRK		00	ff	fa
12	fffe	00	1		0008	f0	4c	e8	fa	nv-BdIzc	BRK		00	ff	ff
13	ffff	00	1		0008	f0	4c	e8	fa	nv-BdIzc	BRK	T0	00	00	ff
13	ffff	00	1		0008	f0	4c	e8	fa	nv-BdIzc	BRK	T0	00	00	00
14	0000	a9	1	LDA #	0000	f0	4c	e8	fa	nv-BdIzc	BRK	T1	ff	00	00
14	0000	a9	1	LDA #	0000	f0	4c	e8	fa	nv-BdIzc	BRK	T1	ff	00	ff
15	0001	f0	1		0001	f0	4c	e8	fa	nv-BdIzc	LDA #	T0+T2	ff	ff	ff
15	0001	f0	1		0001	f0	4c	e8	fa	nv-BdIzc	LDA #	T0+T2	ff	ff	fe

ORA (01)	Cycles: 6	Size: 2
	Indirect X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Operand.
2.1		Inc. **PC**.
2.2	Read Operand	Discard.
3.1		Pointer = u8(Operand + **X**).
3.2	Read Pointer	Store as Address.
4.1		
4.2	Read u8(Pointer + 1)	Store as Address.H.
5.1		
5.2	Read Address	Store as Operand.
6.1		**A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
6.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	01	1	ORA (zp,X)	0006	f0	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	01	1	ORA (zp,X)	0006	f0	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	11	1		0007	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T2	98	98	98
8	0007	11	1		0007	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T2	98	98	30
9	0011	88	1		0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T3	4c	11	30
9	0011	88	1		0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T3	4c	11	5d
10	005d	33	1		0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T4	00	5d	5d
10	005d	33	1		0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T4	00	5d	5e
11	005e	00	1		0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T5	00	33	5e
11	005e	00	1		0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T5	00	33	33
12	0033	55	1		0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T0	ff	ff	33
12	0033	55	1		0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T0	ff	ff	fe
13	0008	10	1	BPL	0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T1	f0	55	fe
13	0008	10	1	BPL	0008	f0	4c	e8	fd	nv-BdIzc	ORA (zp,X)	T1	f0	55	f5
14	0009	00	1		0009	f5	4c	e8	fd	Nv-BdIzc	BPL	T2	f5	f5	f5
14	0009	00	1		0009	f5	4c	e8	fd	Nv-BdIzc	BPL	T2	f5	f5	ea

JAM (02)	Cycles: ∞	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read PC	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read PC	Discard.
2.1		Visual6502 suggests incrementing **PC**, but Single-Step Tests cannot be passed this way.
2.2	Read $FFFF	Discard.
3.1		
3.2	Read $FFFE	Discard.
4.1		
4.2	Read $FFFE	Discard.
5.1		Repeat 5.1 and 5.2 forever.
5.2	Read $FFFF	Discard.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	02	1	unknown	0006	f0	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	02	1	unknown	0006	f0	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	11	1		0007	f0	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	98
8	0007	11	1		0007	f0	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	30
9	ffff	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown	T3	ff	ff	30
9	ffff	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown	T3	ff	ff	fe
10	fffe	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown	T4	ff	ff	fe
10	fffe	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown	T4	ff	ff	fe
11	fffe	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown	T5	ff	ff	fe
11	fffe	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown	T5	ff	ff	fe
12	ffff	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown		ff	ff	fe
12	ffff	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown		ff	ff	fe
13	ffff	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown		ff	ff	fe
13	ffff	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown		ff	ff	fe
14	ffff	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown		ff	ff	fe
14	ffff	00	1		0008	f0	4c	e8	fd	nv-BdIzc	unknown		ff	ff	fe

SLO (03)	Cycles: 8	Size: 2
	Indirect X (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Operand.
2.1		Inc. **PC**.
2.2	Read Operand	Discard.
3.1		Pointer = u8(Operand + **X**).
3.2	Read Pointer	Store as Address.
4.1		
4.2	Read u8(Pointer + 1)	Store as Address.H.
5.1		
5.2	Read Address	Store as Operand.
6.1		
6.2	Write to Address	Write Operand.
7.1		**C** flag = (Operand & $80). Operand <<= 1. **A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
7.2	Write to Address	Write Operand.
8.1		
8.2	Read **PC**	Store as OpCode.
+X.1		* **A** applied.

* Setting of A is delayed by 2 cycles until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	03	1	unknown	0006	00	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	03	1	unknown	0006	00	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	11	1		0007	00	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	98
8	0007	11	1		0007	00	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	30
9	0011	88	1		0008	00	4c	e8	fd	nv-BdIzc	unknown	T3	4c	11	30
9	0011	88	1		0008	00	4c	e8	fd	nv-BdIzc	unknown	T3	4c	11	5d
10	005d	33	1		0008	00	4c	e8	fd	nv-BdIzc	unknown	T4	00	5d	5d
10	005d	33	1		0008	00	4c	e8	fd	nv-BdIzc	unknown	T4	00	5d	5e
11	005e	00	1		0008	00	4c	e8	fd	nv-BdIzc	unknown	T5	00	33	5e
11	005e	00	1		0008	00	4c	e8	fd	nv-BdIzc	unknown	T5	00	33	33
12	0033	c0	1		0008	00	4c	e8	fd	nv-BdIzc	unknown		ff	ff	33
12	0033	c0	1		0008	00	4c	e8	fd	nv-BdIzc	unknown		ff	ff	fe
13	0033	c0	0		0008	00	4c	e8	fd	nv-BdIzc	unknown		c0	c0	fe
13	0033	c0	0		0008	00	4c	e8	fd	nv-BdIzc	unknown		c0	c0	80
14	0033	c0	0		0008	00	4c	e8	fd	Nv-BdIzC	unknown	T0	80	80	80
14	0033	80	0		0008	00	4c	e8	fd	Nv-BdIzC	unknown	T0	80	80	00
15	0008	30	1	BMI	0008	00	4c	e8	fd	Nv-BdIzC	unknown	T1	00	80	00
15	0008	30	1	BMI	0008	00	4c	e8	fd	Nv-BdIzC	unknown	T1	00	80	80
16	0009	00	1		0009	80	4c	e8	fd	Nv-BdIzC	BMI	T2	80	80	80
16	0009	00	1		0009	80	4c	e8	fd	Nv-BdIzC	BMI	T2	80	80	00

NOP (04)	Cycles: 3	Size: 2
	Zero Page (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read Address	Store as Operand.
3.1		
3.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	04	1	unknown	0006	00	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	04	1	unknown	0006	00	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	11	1		0007	00	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	98
8	0007	11	1		0007	00	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	30
9	0011	88	1		0008	00	4c	e8	fd	nv-BdIzc	unknown	T0	ff	11	30
9	0011	88	1		0008	00	4c	e8	fd	nv-BdIzc	unknown	T0	ff	11	10
10	0008	30	1	BMI	0008	00	4c	e8	fd	nv-BdIzc	unknown	T1	ff	88	10
10	0008	30	1	BMI	0008	00	4c	e8	fd	nv-BdIzc	unknown	T1	ff	88	87
11	0009	00	1		0009	00	4c	e8	fd	nv-BdIzc	BMI	T2	87	87	87
11	0009	00	1		0009	00	4c	e8	fd	nv-BdIzc	BMI	T2	87	87	0e

ORA (05)	Cycles: 3	Size: 2
	Zero Page (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read Address	Store as Operand.
3.1		**A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
3.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	05	1	ORA zp	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	05	1	ORA zp	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	11	1		0007	33	4c	e8	fd	nv-BdIzc	ORA zp	T2	98	98	98
8	0007	11	1		0007	33	4c	e8	fd	nv-BdIzc	ORA zp	T2	98	98	30
9	0011	88	1		0008	33	4c	e8	fd	nv-BdIzc	ORA zp	T0	ff	11	30
9	0011	88	1		0008	33	4c	e8	fd	nv-BdIzc	ORA zp	T0	ff	11	10
10	0008	30	1	BMI	0008	33	4c	e8	fd	nv-BdIzc	ORA zp	T1	33	88	10
10	0008	30	1	BMI	0008	33	4c	e8	fd	nv-BdIzc	ORA zp	T1	33	88	bb
11	0009	00	1		0009	bb	4c	e8	fd	Nv-BdIzc	BMI	T2	bb	bb	bb
11	0009	00	1		0009	bb	4c	e8	fd	Nv-BdIzc	BMI	T2	bb	bb	76

ASL (06)	Cycles: 5	Size: 2
	Zero Page (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read Address	Store as Operand.
3.1		
3.2	Write to Address	Write Operand.
4.1		**C** flag = (Operand & $80). Operand <<= 1. **N** flag = (Operand & $80), **Z** flag = !Operand.
4.2	Write to Address	Write Operand.
5.1		
5.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	06	1	ASL zp	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	06	1	ASL zp	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	11	1		0007	33	4c	e8	fd	nv-BdIzc	ASL zp	T2	98	98	98
8	0007	11	1		0007	33	4c	e8	fd	nv-BdIzc	ASL zp	T2	98	98	30
9	0011	7f	1		0008	33	4c	e8	fd	nv-BdIzc	ASL zp	T3	ff	11	30
9	0011	7f	1		0008	33	4c	e8	fd	nv-BdIzc	ASL zp	T3	ff	11	10
10	0011	7f	0		0008	33	4c	e8	fd	nv-BdIzc	ASL zp	T4	7f	7f	10
10	0011	7f	0		0008	33	4c	e8	fd	nv-BdIzc	ASL zp	T4	7f	7f	fe
11	0011	7f	0		0008	33	4c	e8	fd	Nv-BdIzc	ASL zp	T0	fe	fe	fe
11	0011	fe	0		0008	33	4c	e8	fd	Nv-BdIzc	ASL zp	T0	fe	fe	fc
12	0008	30	1	BMI	0008	33	4c	e8	fd	Nv-BdIzc	ASL zp	T1	ff	fe	fc
12	0008	30	1	BMI	0008	33	4c	e8	fd	Nv-BdIzc	ASL zp	T1	ff	fe	fd
13	0009	00	1		0009	33	4c	e8	fd	Nv-BdIzc	BMI	T2	fd	fd	fd
13	0009	00	1		0009	33	4c	e8	fd	Nv-BdIzc	BMI	T2	fd	fd	fa

SLO (07)	Cycles: 5	Size: 2
	Zero Page (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read Address	Store as Operand.
3.1		
3.2	Write to Address	Write Operand.
4.1		**C** flag = (Operand & $80). Operand <<= 1. **A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
4.2	Write to Address	Write Operand.
5.1		
5.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	07	1	unknown	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	07	1	unknown	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	11	1		0007	33	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	98
8	0007	11	1		0007	33	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	30
9	0011	fe	1		0008	33	4c	e8	fd	nv-BdIzc	unknown	T3	ff	11	30
9	0011	fe	1		0008	33	4c	e8	fd	nv-BdIzc	unknown	T3	ff	11	10
10	0011	fe	0		0008	33	4c	e8	fd	nv-BdIzc	unknown	T4	fe	fe	10
10	0011	fe	0		0008	33	4c	e8	fd	nv-BdIzc	unknown	T4	fe	fe	fc
11	0011	fe	0		0008	33	4c	e8	fd	Nv-BdIzC	unknown	T0	fc	fc	fc
11	0011	fc	0		0008	33	4c	e8	fd	Nv-BdIzC	unknown	T0	fc	fc	f8
12	0008	8c	1	STY Abs	0008	33	4c	e8	fd	Nv-BdIzC	unknown	T1	33	fc	f8
12	0008	8c	1	STY Abs	0008	33	4c	e8	fd	Nv-BdIzC	unknown	T1	33	fc	ff
13	0009	44	1		0009	ff	4c	e8	fd	Nv-BdIzC	STY Abs	T2	ff	ff	ff
13	0009	44	1		0009	ff	4c	e8	fd	Nv-BdIzC	STY Abs	T2	ff	ff	fe

PHP (08)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Discard.
2.1		Operand = (**P** | $30).
2.2	Write to u8(**S**+0) | $0100	Write Operand.
3.1		Dec. **S** by 1.
3.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	08	1	PHP	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	08	1	PHP	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	ea	1		0007	33	4c	e8	fd	nv-BdIzc	PHP	T2	98	98	98
8	0007	ea	1		0007	33	4c	e8	fd	nv-BdIzc	PHP	T2	98	98	30
9	01fd	ea	0		0007	33	4c	e8	fd	nv-BdIzc	PHP	T0	ff	fd	30
9	01fd	34	0		0007	33	4c	e8	fd	nv-BdIzc	PHP	T0	ff	fd	fc
10	0007	ea	1	NOP	0007	33	4c	e8	fc	nv-BdIzc	PHP	T1	00	34	fc
10	0007	ea	1	NOP	0007	33	4c	e8	fc	nv-BdIzc	PHP	T1	00	34	34
11	0008	8c	1		0008	33	4c	e8	fc	nv-BdIzc	NOP	T0+T2	34	34	34
11	0008	8c	1		0008	33	4c	e8	fc	nv-BdIzc	NOP	T0+T2	34	34	68

ORA (09)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Operand.
2.1		Inc. **PC**. **A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
2.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	09	1	ORA #	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	09	1	ORA #	0006	33	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	ea	1		0007	33	4c	e8	fd	nv-BdIzc	ORA #	T0+T2	98	98	98
8	0007	ea	1		0007	33	4c	e8	fd	nv-BdIzc	ORA #	T0+T2	98	98	30
9	0008	8c	1	STY Abs	0008	33	4c	e8	fd	nv-BdIzc	ORA #	T1	33	ea	30
9	0008	8c	1	STY Abs	0008	33	4c	e8	fd	nv-BdIzc	ORA #	T1	33	ea	fb
10	0009	44	1		0009	fb	4c	e8	fd	Nv-BdIzc	STY Abs	T2	fb	fb	fb
10	0009	44	1		0009	fb	4c	e8	fd	Nv-BdIzc	STY Abs	T2	fb	fb	f6

ASL (0A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Discard.
2.1		**C** flag = (**A** & $80). **A** <<= 1. **N** flag = (**A** & $80), **Z** flag = !**A**.
2.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	0a	1	ASL	0006	7f	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	0a	1	ASL	0006	7f	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	ea	1		0007	7f	4c	e8	fd	nv-BdIzc	ASL	T0+T2	98	98	98
8	0007	ea	1		0007	7f	4c	e8	fd	nv-BdIzc	ASL	T0+T2	98	98	30
9	0007	ea	1	NOP	0007	7f	4c	e8	fd	nv-BdIzc	ASL	T1	7f	7f	30
9	0007	ea	1	NOP	0007	7f	4c	e8	fd	nv-BdIzc	ASL	T1	7f	7f	fe
10	0008	8c	1		0008	fe	4c	e8	fd	Nv-BdIzc	NOP	T0+T2	fe	fe	fe
10	0008	8c	1		0008	fe	4c	e8	fd	Nv-BdIzc	NOP	T0+T2	fe	fe	fc

ANC (0B)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Operand.
2.1		Inc. **PC**. **A** &= Operand. **C** flag = (**A** & $80), **N** flag = (**A** & $80), **Z** flag = !**A**.
2.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **C** flag, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

Visual6502 omits the update of A; flags are updated based off the incorrect A value.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	0b	1	unknown	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	0b	1	unknown	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	unknown	T0+T2	98	98	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	unknown	T0+T2	98	98	30
9	0008	8c	1	STY Abs	0008	81	4c	e8	fd	nv-BdIzc	unknown	T1	80	80	30
9	0008	8c	1	STY Abs	0008	81	4c	e8	fd	nv-BdIzc	unknown	T1	81	81	81
10	0009	44	1		0009	81	4c	e8	fd	Nv-BdIzC	STY Abs	T2	81	81	81
10	0009	44	1		0009	81	4c	e8	fd	Nv-BdIzC	STY Abs	T2	81	81	02

ANC (ANC2, ANA, ANB)

Type: Combination of an immediate and an implied command (Sub-instructions: AND, ASL/ROL)

Opc.	Mnemonic	Function	Size	Cycles	N	V	-	B	D	I	Z	C
$0B	ANC #imm	A = A & #{imm}	2	2	o						o	o
$2B	ANC #imm	A = A & #{imm}	2	2	o						o	o

Operation: ANDs the contents of the A register with an immediate value and then moves bit 7 of A into the Carry flag.

- This opcode works basically identically to AND #imm. except that the Carry flag is set to the same state that the Negative flag is set to. (bit 7 is put into the carry, as if the ASL/ROL would have been executed)

— NoMoreSecrets-NMOS6510UnintendedOpcodes-20232412.pdf

NOP (0C)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	0c	1	unknown	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	0c	1	unknown	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	unknown	T2	98	98	30
9	0008	b8	1		0008	81	4c	e8	fd	nv-BdIzc	unknown	T3	00	f0	30
9	0008	b8	1		0008	81	4c	e8	fd	nv-BdIzc	unknown	T3	00	f0	f0
10	b8f0	00	1		0009	81	4c	e8	fd	nv-BdIzc	unknown	T0	ff	f0	f0
10	b8f0	00	1		0009	81	4c	e8	fd	nv-BdIzc	unknown	T0	ff	f0	ef
11	0009	b8	1	CLV	0009	81	4c	e8	fd	nv-BdIzc	unknown	T1	ff	00	ef
11	0009	b8	1	CLV	0009	81	4c	e8	fd	nv-BdIzc	unknown	T1	ff	00	ff
12	000a	15	1		000a	81	4c	e8	fd	nv-BdIzc	CLV	T0+T2	ff	ff	ff
12	000a	15	1		000a	81	4c	e8	fd	nv-BdIzc	CLV	T0+T2	ff	ff	fe

ORA (0D)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address	Store as Operand.
4.1		**A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
4.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	0d	1	ORA Abs	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	0d	1	ORA Abs	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	ORA Abs	T2	98	98	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	ORA Abs	T2	98	98	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	ORA Abs	T3	00	f0	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	ORA Abs	T3	00	f0	f0
10	01f0	aa	1		0009	81	4c	e8	fd	nv-BdIzc	ORA Abs	T0	ff	ff	f0
10	01f0	aa	1		0009	81	4c	e8	fd	nv-BdIzc	ORA Abs	T0	ff	ff	fe
11	0009	b8	1	CLV	0009	81	4c	e8	fd	nv-BdIzc	ORA Abs	T1	81	aa	fe
11	0009	b8	1	CLV	0009	81	4c	e8	fd	nv-BdIzc	ORA Abs	T1	81	aa	ab
12	000a	15	1		000a	ab	4c	e8	fd	Nv-BdIzc	CLV	T0+T2	ab	ab	ab
12	000a	15	1		000a	ab	4c	e8	fd	Nv-BdIzc	CLV	T0+T2	ab	ab	56

ASL (0E)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.
5.1		**C** flag = (Operand & $80). Operand <<= 1. **N** flag = (Operand & $80), **Z** flag = !Operand.
5.2	Write to Address	Write Operand.
6.1		
6.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	0e	1	ASL Abs	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	0e	1	ASL Abs	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	ASL Abs	T2	98	98	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	ASL Abs	T2	98	98	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	ASL Abs	T3	00	f0	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	ASL Abs	T3	00	f0	f0
10	01f0	7f	1		0009	81	4c	e8	fd	nv-BdIzc	ASL Abs	T4	ff	ff	f0
10	01f0	7f	1		0009	81	4c	e8	fd	nv-BdIzc	ASL Abs	T4	ff	ff	fe
11	01f0	7f	0		0009	81	4c	e8	fd	nv-BdIzc	ASL Abs	T5	7f	7f	fe
11	01f0	7f	0		0009	81	4c	e8	fd	nv-BdIzc	ASL Abs	T5	7f	7f	fe
12	01f0	7f	0		0009	81	4c	e8	fd	Nv-BdIzc	ASL Abs	T0	fe	fe	fe
12	01f0	fe	0		0009	81	4c	e8	fd	Nv-BdIzc	ASL Abs	T0	fe	fe	fc
13	0009	b8	1	CLV	0009	81	4c	e8	fd	Nv-BdIzc	ASL Abs	T1	ff	fe	fc
13	0009	b8	1	CLV	0009	81	4c	e8	fd	Nv-BdIzc	ASL Abs	T1	ff	fe	fd
14	000a	15	1		000a	81	4c	e8	fd	Nv-BdIzc	CLV	T0+T2	fd	fd	fd
14	000a	15	1		000a	81	4c	e8	fd	Nv-BdIzc	CLV	T0+T2	fd	fd	fa

SLO (0F)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.
5.1		**C** flag = (Operand & $80). Operand <<= 1. **A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
5.2	Write to Address	Write Operand.
6.1		
6.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	0e	1	ASL Abs	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	0e	1	ASL Abs	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	ASL Abs	T2	98	98	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	ASL Abs	T2	98	98	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	ASL Abs	T3	00	f0	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	ASL Abs	T3	00	f0	f0
10	01f0	7f	1		0009	81	4c	e8	fd	nv-BdIzc	ASL Abs	T4	ff	ff	f0
10	01f0	7f	1		0009	81	4c	e8	fd	nv-BdIzc	ASL Abs	T4	ff	ff	fe
11	01f0	7f	0		0009	81	4c	e8	fd	nv-BdIzc	ASL Abs	T5	7f	7f	fe
11	01f0	7f	0		0009	81	4c	e8	fd	nv-BdIzc	ASL Abs	T5	7f	7f	fe
12	01f0	7f	0		0009	81	4c	e8	fd	Nv-BdIzc	ASL Abs	T0	fe	fe	fe
12	01f0	fe	0		0009	81	4c	e8	fd	Nv-BdIzc	ASL Abs	T0	fe	fe	fc
13	0009	b8	1	CLV	0009	81	4c	e8	fd	Nv-BdIzc	ASL Abs	T1	ff	fe	fc
13	0009	b8	1	CLV	0009	81	4c	e8	fd	Nv-BdIzc	ASL Abs	T1	ff	fe	fd
14	000a	15	1		000a	81	4c	e8	fd	Nv-BdIzc	CLV	T0+T2	fd	fd	fd
14	000a	15	1		000a	81	4c	e8	fd	Nv-BdIzc	CLV	T0+T2	fd	fd	fa

BPL (10)	Cycles: 2-4	Size: 2
	Relative (Jump)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**. Jump = ((**P** & $80) == $00).
1.2	Read **PC**	Store as Operand.
		If !Jump, poll interrupts.
		Otherwise Address = i16(i8(Operand)) + **PC** + 1.
		BoundaryCrossed = Address.H != **PC**.H.
		If !BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If !Jump, end (next half-cycle is 4.2).
2.2	Read **PC**	Discard.
3.1		**PC**.L = Address.L. If !BoundaryCrossed, end (next half-cycle is 4.2).
3.2	Read **PC**	Discard. Poll interrupts.
4.1		**PC**.H = Address.H.
4.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	10	1	BPL	0006	81	8c	e8	fd	Nv-BdIzc	LDX #	T1	8c	8c	a0
7	0006	10	1	BPL	0006	81	8c	e8	fd	Nv-BdIzc	LDX #	T1	8c	8c	18
8	0007	10	1		0007	81	8c	e8	fd	Nv-BdIzc	BPL	T2	18	18	18
8	0007	10	1		0007	81	8c	e8	fd	Nv-BdIzc	BPL	T2	18	18	30
9	0008	01	1	ORA (zp,X)	0008	81	8c	e8	fd	Nv-BdIzc	BPL		10	08	30
9	0008	01	1	ORA (zp,X)	0008	81	8c	e8	fd	Nv-BdIzc	BPL		10	08	18
10	0009	b8	1		0009	81	8c	e8	fd	Nv-BdIzc	ORA (zp,X)	T2	18	18	18
10	0009	b8	1		0009	81	8c	e8	fd	Nv-BdIzc	ORA (zp,X)	T2	18	18	30

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	10	1	BPL	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	10	1	BPL	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	10	1		0007	81	4c	e8	fd	nv-BdIzc	BPL	T2	98	98	98
8	0007	10	1		0007	81	4c	e8	fd	nv-BdIzc	BPL	T2	98	98	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	BPL	T3	10	08	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	BPL	T3	10	08	18
10	0018	48	1	PHA	0018	81	4c	e8	fd	nv-BdIzc	BPL		00	00	18
10	0018	48	1	PHA	0018	81	4c	e8	fd	nv-BdIzc	BPL		00	00	01
11	0019	ea	1		0019	81	4c	e8	fd	nv-BdIzc	PHA	T2	01	01	01
11	0019	ea	1		0019	81	4c	e8	fd	nv-BdIzc	PHA	T2	01	01	02

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	10	1	BPL	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	a0
7	0006	10	1	BPL	0006	81	4c	e8	fd	nv-BdIzc	LDX #	T1	4c	4c	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	BPL	T2	98	98	98
8	0007	f0	1		0007	81	4c	e8	fd	nv-BdIzc	BPL	T2	98	98	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	BPL	T3	f0	08	30
9	0008	01	1		0008	81	4c	e8	fd	nv-BdIzc	BPL	T3	f0	08	f8
10	00f8	00	1		00f8	81	4c	e8	fd	nv-BdIzc	BPL	T0	00	ff	f8
10	00f8	00	1		00f8	81	4c	e8	fd	nv-BdIzc	BPL	T0	00	ff	ff
11	fff8	00	1	BRK	fff8	81	4c	e8	fd	nv-BdIzc	BPL	T1	ff	00	ff
11	fff8	00	1	BRK	fff8	81	4c	e8	fd	nv-BdIzc	BPL	T1	ff	00	ff
12	fff9	00	1		fff9	81	4c	e8	fd	nv-BdIzc	BRK	T2	ff	ff	ff
12	fff9	00	1		fff9	81	4c	e8	fd	nv-BdIzc	BRK	T2	ff	ff	fe

ORA (11)	Cycles: 5-6	Size: 2
	Indirect Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read Pointer	Store as Address.
3.1		
3.2	Read u8(Pointer + 1)	Store as Address.H.
4.1		Target = Address + Y.
		Pointer.L = Target.L.
		Pointer.H = Address.H.
		BoundaryCrossed = Pointer.H != Target.H.
4.2	Read Pointer	Store as Operand. If BoundaryCrossed, skip the next cycle.
5.1		Pointer.H = Target.H.
5.2	Read Pointer	Store as Operand.
6.1		**A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
6.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	11	1	ORA (zp),Y	0006	59	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	7c
7	0006	11	1	ORA (zp),Y	0006	59	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	18
8	0007	20	1		0007	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T2	18	18	18
8	0007	20	1		0007	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T2	18	18	30
9	0020	23	1		0008	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T3	00	20	30
9	0020	23	1		0008	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T3	00	20	21
10	0021	01	1		0008	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T4	1f	23	21
10	0021	01	1		0008	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T4	1f	23	42
11	0142	33	1		0008	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T0	00	01	42
11	0142	33	1		0008	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T0	00	01	02
12	0008	ea	1	NOP	0008	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T1	59	33	02
12	0008	ea	1	NOP	0008	59	8c	1f	fd	Nv-BdIzc	ORA (zp),Y	T1	59	33	7b
13	0009	b8	1		0009	7b	8c	1f	fd	nv-BdIzc	NOP	T0+T2	7b	7b	7b
13	0009	b8	1		0009	7b	8c	1f	fd	nv-BdIzc	NOP	T0+T2	7b	7b	f6

JAM (12)	Cycles: ∞	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read PC	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read PC	Discard.
2.1		Visual6502 suggests incrementing **PC**, but Single-Step Tests cannot be passed this way.
2.2	Read $FFFF	Discard.
3.1		
3.2	Read $FFFE	Discard.
4.1		
4.2	Read $FFFE	Discard.
5.1		Repeat 5.1 and 5.2 forever.
5.2	Read $FFFF	Discard.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	12	1	unknown	0006	59	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	7c
7	0006	12	1	unknown	0006	59	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	18
8	0007	20	1		0007	59	8c	1f	fd	Nv-BdIzc	unknown	T2	18	18	18
8	0007	20	1		0007	59	8c	1f	fd	Nv-BdIzc	unknown	T2	18	18	30
9	ffff	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T3	ff	ff	30
9	ffff	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T3	ff	ff	fe
10	fffe	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T4	ff	ff	fe
10	fffe	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T4	ff	ff	fe
11	fffe	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T5	ff	ff	fe
11	fffe	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T5	ff	ff	fe
12	ffff	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown		ff	ff	fe
12	ffff	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown		ff	ff	fe
13	ffff	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown		ff	ff	fe
13	ffff	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown		ff	ff	fe
14	ffff	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown		ff	ff	fe
14	ffff	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown		ff	ff	fe

SLO (13)	Cycles: 8	Size: 2
	Indirect Y (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read Pointer	Store as Address.
3.1		
3.2	Read u8(Pointer + 1)	Store as Address.H.
4.1		Target = Address + **Y**.
		Pointer.L = Target.L.
		Pointer.H = Address.H.
		BoundaryCrossed = Pointer.H != Target.H.
4.2	Read Pointer	Store as Operand.
5.1		Pointer.H = Target.H.
5.2	Read Pointer	Store as Operand.
6.1		
6.2	Write to Pointer	Write Operand.
7.1		**C** flag = (Operand & $80).
		Operand <<= 1.
		A |= Operand.
		N flag = (**A** & $80), **Z** flag = !**A**.
7.2	Write to Pointer	Write Operand.
8.1		
8.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	13	1	unknown	0006	59	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	7c
7	0006	13	1	unknown	0006	59	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	18
8	0007	20	1		0007	59	8c	1f	fd	Nv-BdIzc	unknown	T2	18	18	18
8	0007	20	1		0007	59	8c	1f	fd	Nv-BdIzc	unknown	T2	18	18	30
9	0020	23	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T3	00	20	30
9	0020	23	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T3	00	20	21
10	0021	01	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T4	1f	23	21
10	0021	01	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T4	1f	23	42
11	0142	33	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T5	00	01	42
11	0142	33	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T5	00	01	02
12	0142	33	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown		02	ff	02
12	0142	33	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown		02	ff	01
13	0142	33	0		0008	59	8c	1f	fd	Nv-BdIzc	unknown		33	33	01
13	0142	33	0		0008	59	8c	1f	fd	Nv-BdIzc	unknown		33	33	66
14	0142	33	0		0008	59	8c	1f	fd	nv-BdIzc	unknown	T0	66	66	66
14	0142	66	0		0008	59	8c	1f	fd	nv-BdIzc	unknown	T0	66	66	cc
15	0008	ea	1	NOP	0008	59	8c	1f	fd	nv-BdIzc	unknown	T1	59	66	cc
15	0008	ea	1	NOP	0008	59	8c	1f	fd	nv-BdIzc	unknown	T1	59	66	7f
16	0009	b8	1		0009	7f	8c	1f	fd	nv-BdIzc	NOP	T0+T2	7f	7f	7f
16	0009	b8	1		0009	7f	8c	1f	fd	nv-BdIzc	NOP	T0+T2	7f	7f	fe

NOP (14)	Cycles: 4	Size: 2
	Zero Page X-Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read Pointer	Store as Operand.
3.1		Address = u8(Pointer + **X**).
3.2	Read Address	Store as Operand.
4.1		
4.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	14	1	unknown	0006	59	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	7c
7	0006	14	1	unknown	0006	59	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	18
8	0007	20	1		0007	59	8c	1f	fd	Nv-BdIzc	unknown	T2	18	18	18
8	0007	20	1		0007	59	8c	1f	fd	Nv-BdIzc	unknown	T2	18	18	30
9	0020	23	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T3	8c	20	30
9	0020	23	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T3	8c	20	ac
10	00ac	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T0	ff	ff	ac
10	00ac	00	1		0008	59	8c	1f	fd	Nv-BdIzc	unknown	T0	ff	ff	fe
11	0008	ea	1	NOP	0008	59	8c	1f	fd	Nv-BdIzc	unknown	T1	ff	00	fe
11	0008	ea	1	NOP	0008	59	8c	1f	fd	Nv-BdIzc	unknown	T1	ff	00	ff
12	0009	b8	1		0009	59	8c	1f	fd	Nv-BdIzc	NOP	T0+T2	ff	ff	ff
12	0009	b8	1		0009	59	8c	1f	fd	Nv-BdIzc	NOP	T0+T2	ff	ff	fe

ORA (15)	Cycles: 4	Size: 2
	Zero Page X-Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read Pointer	Store as Operand.
3.1		Address = u8(Pointer + **X**).
3.2	Read Address	Store as Operand.
4.1		**A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
4.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	15	1	ORA zp,X	0006	70	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	7c
7	0006	15	1	ORA zp,X	0006	70	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	18
8	0007	0f	1		0007	70	8c	1f	fd	Nv-BdIzc	ORA zp,X	T2	18	18	18
8	0007	0f	1		0007	70	8c	1f	fd	Nv-BdIzc	ORA zp,X	T2	18	18	30
9	000f	41	1		0008	70	8c	1f	fd	Nv-BdIzc	ORA zp,X	T3	8c	0f	30
9	000f	41	1		0008	70	8c	1f	fd	Nv-BdIzc	ORA zp,X	T3	8c	0f	9b
10	009b	33	1		0008	70	8c	1f	fd	Nv-BdIzc	ORA zp,X	T0	ff	ff	9b
10	009b	33	1		0008	70	8c	1f	fd	Nv-BdIzc	ORA zp,X	T0	ff	ff	fe
11	0008	ea	1	NOP	0008	70	8c	1f	fd	Nv-BdIzc	ORA zp,X	T1	70	33	fe
11	0008	ea	1	NOP	0008	70	8c	1f	fd	Nv-BdIzc	ORA zp,X	T1	70	33	73
12	0009	b8	1		0009	73	8c	1f	fd	nv-BdIzc	NOP	T0+T2	73	73	73
12	0009	b8	1		0009	73	8c	1f	fd	nv-BdIzc	NOP	T0+T2	73	73	e6

ASL (16)	Cycles: 6	Size: 2
	Zero Page X-Indexed (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read Pointer	Store as Operand.
3.1		Address = u8(Pointer + **X**).
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.
5.1		**C** flag = (Operand & $80). Operand <<= 1. **N** flag = (Operand & $80), **Z** flag = !Operand.
5.2	Write to Address	Write Operand.
6.1		
6.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	16	1	ASL zp,X	0006	70	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	7c
7	0006	16	1	ASL zp,X	0006	70	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	18
8	0007	0f	1		0007	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T2	18	18	18
8	0007	0f	1		0007	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T2	18	18	30
9	000f	41	1		0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T3	8c	0f	30
9	000f	41	1		0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T3	8c	0f	9b
10	009b	66	1		0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T4	ff	ff	9b
10	009b	66	1		0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T4	ff	ff	fe
11	009b	66	0		0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T5	66	66	fe
11	009b	66	0		0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T5	66	66	cc
12	009b	66	0		0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T0	cc	cc	cc
12	009b	cc	0		0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T0	cc	cc	98
13	0008	ea	1	NOP	0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T1	ff	cc	98
13	0008	ea	1	NOP	0008	70	8c	1f	fd	Nv-BdIzc	ASL zp,X	T1	ff	cc	cb
14	0009	b8	1		0009	70	8c	1f	fd	Nv-BdIzc	NOP	T0+T2	cb	cb	cb
14	0009	b8	1		0009	70	8c	1f	fd	Nv-BdIzc	NOP	T0+T2	cb	cb	96

SLO (17)	Cycles: 6	Size: 2
	Zero Page X-Indexed (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read Pointer	Store as Operand.
3.1		Address = u8(Pointer + **X**).
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.
5.1		**C** flag = (Operand & $80).
		Operand <<= 1.
		A |= Operand.
		N flag = (**A** & $80), **Z** flag = !**A**.
5.2	Write to Address	Write Operand.
6.1		
6.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	17	1	unknown	0006	70	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	7c
7	0006	17	1	unknown	0006	70	8c	1f	fd	Nv-BdIzc	LDX #	T1	8c	8c	18
8	0007	0f	1		0007	70	8c	1f	fd	Nv-BdIzc	unknown	T2	18	18	18
8	0007	0f	1		0007	70	8c	1f	fd	Nv-BdIzc	unknown	T2	18	18	30
9	000f	41	1		0008	70	8c	1f	fd	Nv-BdIzc	unknown	T3	8c	0f	30
9	000f	41	1		0008	70	8c	1f	fd	Nv-BdIzc	unknown	T3	8c	0f	9b
10	009b	33	1		0008	70	8c	1f	fd	Nv-BdIzc	unknown	T4	ff	ff	9b
10	009b	33	1		0008	70	8c	1f	fd	Nv-BdIzc	unknown	T4	ff	ff	fe
11	009b	33	0		0008	70	8c	1f	fd	Nv-BdIzc	unknown	T5	33	33	fe
11	009b	33	0		0008	70	8c	1f	fd	Nv-BdIzc	unknown	T5	33	33	66
12	009b	33	0		0008	70	8c	1f	fd	nv-BdIzc	unknown	T0	66	66	66
12	009b	66	0		0008	70	8c	1f	fd	nv-BdIzc	unknown	T0	66	66	cc
13	0008	ea	1	NOP	0008	70	8c	1f	fd	nv-BdIzc	unknown	T1	70	66	cc
13	0008	ea	1	NOP	0008	70	8c	1f	fd	nv-BdIzc	unknown	T1	70	66	76
14	0009	b8	1		0009	76	8c	1f	fd	nv-BdIzc	NOP	T0+T2	76	76	76
14	0009	b8	1		0009	76	8c	1f	fd	nv-BdIzc	NOP	T0+T2	76	76	ec

CLC (18)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Discard.
2.1		**C** flag = 0.
2.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
9	0006	18	1	CLC	0006	aa	8c	1f	fd	Nv-BdIzC	LDX #	T1	8c	8c	7c
9	0006	18	1	CLC	0006	aa	8c	1f	fd	Nv-BdIzC	LDX #	T1	8c	8c	18
10	0007	ea	1		0007	aa	8c	1f	fd	Nv-BdIzC	CLC	T0+T2	18	18	18
10	0007	ea	1		0007	aa	8c	1f	fd	Nv-BdIzC	CLC	T0+T2	18	18	30
11	0007	ea	1	NOP	0007	aa	8c	1f	fd	Nv-BdIzc	CLC	T1	ff	ff	30
11	0007	ea	1	NOP	0007	aa	8c	1f	fd	Nv-BdIzc	CLC	T1	ff	ff	fe
12	0008	ea	1		0008	aa	8c	1f	fd	Nv-BdIzc	NOP	T0+T2	fe	fe	fe
12	0008	ea	1		0008	aa	8c	1f	fd	Nv-BdIzc	NOP	T0+T2	fe	fe	fc

ORA (19)	Cycles: 4-5	Size: 3
	Absolute Y-Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Pointer.H.
3.1		Inc. **PC**. Target = Pointer + **Y**.
		Address.L = Target.L.
		Address.H = Pointer.H.
		BoundaryCrossed = Pointer.H != Target.H.
3.2	Read Address	Store as Operand. If BoundaryCrossed, skip the next cycle.
4.1		Address.H = Target.H.
4.2	Read Address	Store as Operand.
5.1		**A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
5.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	19	1	ORA Abs,Y	0006	ea	11	1f	fd	nv-BdIzc	LDX #	T1	11	11	7c
7	0006	19	1	ORA Abs,Y	0006	ea	11	1f	fd	nv-BdIzc	LDX #	T1	11	11	22
8	0007	04	1		0007	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T2	22	22	22
8	0007	04	1		0007	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T2	22	22	44
9	0008	00	1		0008	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T3	1f	04	44
9	0008	00	1		0008	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T3	1f	04	23
10	0023	f0	1		0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T0	00	00	23
10	0023	f0	1		0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T0	00	00	01
11	0009	b8	1	CLV	0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T1	ea	f0	01
11	0009	b8	1	CLV	0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T1	ea	f0	fa
12	000a	15	1		000a	fa	11	1f	fd	Nv-BdIzc	CLV	T0+T2	fa	fa	fa
12	000a	15	1		000a	fa	11	1f	fd	Nv-BdIzc	CLV	T0+T2	fa	fa	f4

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	19	1	ORA Abs,Y	0006	ea	11	1f	fd	nv-BdIzc	LDX #	T1	11	11	7c
7	0006	19	1	ORA Abs,Y	0006	ea	11	1f	fd	nv-BdIzc	LDX #	T1	11	11	22
8	0007	f4	1		0007	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T2	22	22	22
8	0007	f4	1		0007	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T2	22	22	44
9	0008	00	1		0008	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T3	1f	f4	44
9	0008	00	1		0008	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T3	1f	f4	13
10	0013	0f	1		0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T4	00	00	13
10	0013	0f	1		0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T4	00	00	01
11	0113	01	1		0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T0	01	ff	01
11	0113	01	1		0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T0	01	ff	00
12	0009	b8	1	CLV	0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T1	ea	01	00
12	0009	b8	1	CLV	0009	ea	11	1f	fd	nv-BdIzc	ORA Abs,Y	T1	ea	01	eb
13	000a	15	1		000a	eb	11	1f	fd	Nv-BdIzc	CLV	T0+T2	eb	eb	eb
13	000a	15	1		000a	eb	11	1f	fd	Nv-BdIzc	CLV	T0+T2	eb	eb	d6

NOP (1A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Discard.
2.1		
2.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	1a	1	unknown	0006	ea	11	1f	fd	nv-BdIzc	LDX #	T1	11	11	7c
7	0006	1a	1	unknown	0006	ea	11	1f	fd	nv-BdIzc	LDX #	T1	11	11	22
8	0007	ea	1		0007	ea	11	1f	fd	nv-BdIzc	unknown	T0+T2	22	22	22
8	0007	ea	1		0007	ea	11	1f	fd	nv-BdIzc	unknown	T0+T2	22	22	44
9	0007	ea	1	NOP	0007	ea	11	1f	fd	nv-BdIzc	unknown	T1	ff	ff	44
9	0007	ea	1	NOP	0007	ea	11	1f	fd	nv-BdIzc	unknown	T1	ff	ff	fe
10	0008	00	1		0008	ea	11	1f	fd	nv-BdIzc	NOP	T0+T2	fe	fe	fe
10	0008	00	1		0008	ea	11	1f	fd	nv-BdIzc	NOP	T0+T2	fe	fe	fc

SLO (1B)	Cycles: 7	Size: 3
	Absolute Y-Indexed (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Pointer.H.
3.1		Inc. **PC**. Target = Pointer + **Y**.
		Address.L = Target.L.
		Address.H = Pointer.H.
		BoundaryCrossed = Pointer.H != Target.H.
3.2	Read Address	Store as Operand.
4.1		Address.H = Target.H.
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.
6.1		**C** flag = (Operand & $80).
		Operand <<= 1.
		A |= Operand.
		N flag = (**A** & $80), **Z** flag = !**A**.
6.2	Write to Address	Write Operand.
7.1		
7.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	1b	1	unknown	0006	ea	11	1f	fd	nv-BdIzc	LDX #	T1	11	11	7c
7	0006	1b	1	unknown	0006	ea	11	1f	fd	nv-BdIzc	LDX #	T1	11	11	22
8	0007	44	1		0007	ea	11	1f	fd	nv-BdIzc	unknown	T2	22	22	22
8	0007	44	1		0007	ea	11	1f	fd	nv-BdIzc	unknown	T2	22	22	44
9	0008	01	1		0008	ea	11	1f	fd	nv-BdIzc	unknown	T3	1f	44	44
9	0008	01	1		0008	ea	11	1f	fd	nv-BdIzc	unknown	T3	1f	44	63
10	0163	55	1		0009	ea	11	1f	fd	nv-BdIzc	unknown	T4	00	01	63
10	0163	55	1		0009	ea	11	1f	fd	nv-BdIzc	unknown	T4	00	01	02
11	0163	55	1		0009	ea	11	1f	fd	nv-BdIzc	unknown	T5	02	ff	02
11	0163	55	1		0009	ea	11	1f	fd	nv-BdIzc	unknown	T5	02	ff	01
12	0163	55	0		0009	ea	11	1f	fd	nv-BdIzc	unknown		55	55	01
12	0163	55	0		0009	ea	11	1f	fd	nv-BdIzc	unknown		55	55	aa
13	0163	55	0		0009	ea	11	1f	fd	Nv-BdIzc	unknown	T0	aa	aa	aa
13	0163	aa	0		0009	ea	11	1f	fd	Nv-BdIzc	unknown	T0	aa	aa	54
14	0009	b8	1	CLV	0009	ea	11	1f	fd	Nv-BdIzc	unknown	T1	ea	aa	54
14	0009	b8	1	CLV	0009	ea	11	1f	fd	Nv-BdIzc	unknown	T1	ea	aa	ea
15	000a	15	1		000a	ea	11	1f	fd	Nv-BdIzc	CLV	T0+T2	ea	ea	ea
15	000a	15	1		000a	ea	11	1f	fd	Nv-BdIzc	CLV	T0+T2	ea	ea	d4

NOP (1C)	Cycles: 4-5	Size: 3
	Absolute X-Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Pointer.H.
3.1		Inc. **PC**. Target = Pointer + **X**.
		Address.L = Target.L.
		Address.H = Pointer.H.
		BoundaryCrossed = Pointer.H != Target.H.
3.2	Read Address	Store as Operand. If BoundaryCrossed, skip the next cycle.
4.1		Address.H = Target.H.
4.2	Read Address	Store as Operand.
5.1		
5.2	**Read PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	1c	1	unknown	0006	ea	fe	1f	fd	Nv-BdIzc	LDX #	T1	fe	fe	7c
7	0006	1c	1	unknown	0006	ea	fe	1f	fd	Nv-BdIzc	LDX #	T1	fe	fe	fc
8	0007	44	1		0007	ea	fe	1f	fd	Nv-BdIzc	unknown	T2	fc	fc	fc
8	0007	44	1		0007	ea	fe	1f	fd	Nv-BdIzc	unknown	T2	fc	fc	f8
9	0008	01	1		0008	ea	fe	1f	fd	Nv-BdIzc	unknown	T3	fe	44	f8
9	0008	01	1		0008	ea	fe	1f	fd	Nv-BdIzc	unknown	T3	fe	44	42
10	0142	66	1		0009	ea	fe	1f	fd	Nv-BdIzc	unknown	T4	00	01	42
10	0142	66	1		0009	ea	fe	1f	fd	Nv-BdIzc	unknown	T4	00	01	02
11	0242	00	1		0009	ea	fe	1f	fd	Nv-BdIzc	unknown	T0	02	ff	02
11	0242	00	1		0009	ea	fe	1f	fd	Nv-BdIzc	unknown	T0	02	ff	01
12	0009	b8	1	CLV	0009	ea	fe	1f	fd	Nv-BdIzc	unknown	T1	ff	00	01
12	0009	b8	1	CLV	0009	ea	fe	1f	fd	Nv-BdIzc	unknown	T1	ff	00	ff
13	000a	15	1		000a	ea	fe	1f	fd	Nv-BdIzc	CLV	T0+T2	ff	ff	ff
13	000a	15	1		000a	ea	fe	1f	fd	Nv-BdIzc	CLV	T0+T2	ff	ff	fe

ORA (1D)	Cycles: 4-5	Size: 3
	Absolute X-Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Pointer.H.
3.1		Inc. **PC**. Target = Pointer + **X**.
		Address.L = Target.L.
		Address.H = Pointer.H.
		BoundaryCrossed = Pointer.H != Target.H.
3.2	Read Address	Store as Operand. If BoundaryCrossed, skip the next cycle.
4.1		Address.H = Target.H.
4.2	Read Address	Store as Operand.
5.1		**A** |= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
5.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	1d	1	ORA Abs,X	0006	ea	14	1f	fd	nv-BdIzc	LDX #	T1	14	14	7c
7	0006	1d	1	ORA Abs,X	0006	ea	14	1f	fd	nv-BdIzc	LDX #	T1	14	14	28
8	0007	44	1		0007	ea	14	1f	fd	nv-BdIzc	ORA Abs,X	T2	28	28	28
8	0007	44	1		0007	ea	14	1f	fd	nv-BdIzc	ORA Abs,X	T2	28	28	50
9	0008	01	1		0008	ea	14	1f	fd	nv-BdIzc	ORA Abs,X	T3	14	44	50
9	0008	01	1		0008	ea	14	1f	fd	nv-BdIzc	ORA Abs,X	T3	14	44	58
10	0158	00	1		0009	ea	14	1f	fd	nv-BdIzc	ORA Abs,X	T0	00	01	58
10	0158	00	1		0009	ea	14	1f	fd	nv-BdIzc	ORA Abs,X	T0	00	01	02
11	0009	b8	1	CLV	0009	ea	14	1f	fd	nv-BdIzc	ORA Abs,X	T1	ea	00	02
11	0009	b8	1	CLV	0009	ea	14	1f	fd	nv-BdIzc	ORA Abs,X	T1	ea	00	ea
12	000a	15	1		000a	ea	14	1f	fd	Nv-BdIzc	CLV	T0+T2	ea	ea	ea
12	000a	15	1		000a	ea	14	1f	fd	Nv-BdIzc	CLV	T0+T2	ea	ea	d4

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	1d	1	ORA Abs,X	0006	ea	fc	1f	fd	Nv-BdIzc	LDX #	T1	fc	fc	7c
7	0006	1d	1	ORA Abs,X	0006	ea	fc	1f	fd	Nv-BdIzc	LDX #	T1	fc	fc	f8
8	0007	44	1		0007	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T2	f8	f8	f8
8	0007	44	1		0007	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T2	f8	f8	f0
9	0008	01	1		0008	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T3	fc	44	f0
9	0008	01	1		0008	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T3	fc	44	40
10	0140	00	1		0009	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T4	00	01	40
10	0140	00	1		0009	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T4	00	01	02
11	0240	00	1		0009	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T0	02	ff	02
11	0240	00	1		0009	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T0	02	ff	01
12	0009	b8	1	CLV	0009	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T1	ea	00	01
12	0009	b8	1	CLV	0009	ea	fc	1f	fd	Nv-BdIzc	ORA Abs,X	T1	ea	00	ea
13	000a	15	1		000a	ea	fc	1f	fd	Nv-BdIzc	CLV	T0+T2	ea	ea	ea
13	000a	15	1		000a	ea	fc	1f	fd	Nv-BdIzc	CLV	T0+T2	ea	ea	d4

ASL (1E)	Cycles: 7	Size: 3
	Absolute X-Indexed (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Pointer.H.
3.1		Inc. **PC**. Target = Pointer + **X**.
		Address.L = Target.L.
		Address.H = Pointer.H.
		BoundaryCrossed = Pointer.H != Target.H.
3.2	Read Address	Store as Operand.
4.1		Address.H = Target.H.
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.
6.1		**C** flag = (Operand & $80). Operand <<= 1. **N** flag = (Operand & $80), **Z** flag = !Operand.
6.2	Write to Address	Write Operand.
7.1		
7.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	1e	1	ASL Abs,X	0006	ea	13	1f	fd	nv-BdIzc	LDX #	T1	13	13	7c
7	0006	1e	1	ASL Abs,X	0006	ea	13	1f	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	ea	13	1f	fd	nv-BdIzc	ASL Abs,X	T2	26	26	26
8	0007	44	1		0007	ea	13	1f	fd	nv-BdIzc	ASL Abs,X	T2	26	26	4c
9	0008	01	1		0008	ea	13	1f	fd	nv-BdIzc	ASL Abs,X	T3	13	44	4c
9	0008	01	1		0008	ea	13	1f	fd	nv-BdIzc	ASL Abs,X	T3	13	44	57
10	0157	71	1		0009	ea	13	1f	fd	nv-BdIzc	ASL Abs,X	T4	00	01	57
10	0157	71	1		0009	ea	13	1f	fd	nv-BdIzc	ASL Abs,X	T4	00	01	02
11	0157	71	1		0009	ea	13	1f	fd	nv-BdIzc	ASL Abs,X	T5	02	ff	02
11	0157	71	1		0009	ea	13	1f	fd	nv-BdIzc	ASL Abs,X	T5	02	ff	01
12	0157	71	0		0009	ea	13	1f	fd	nv-BdIzc	ASL Abs,X		71	71	01
12	0157	71	0		0009	ea	13	1f	fd	nv-BdIzc	ASL Abs,X		71	71	e2
13	0157	71	0		0009	ea	13	1f	fd	Nv-BdIzc	ASL Abs,X	T0	e2	e2	e2
13	0157	e2	0		0009	ea	13	1f	fd	Nv-BdIzc	ASL Abs,X	T0	e2	e2	c4
14	0009	b8	1	CLV	0009	ea	13	1f	fd	Nv-BdIzc	ASL Abs,X	T1	ff	e2	c4
14	0009	b8	1	CLV	0009	ea	13	1f	fd	Nv-BdIzc	ASL Abs,X	T1	ff	e2	e1
15	000a	15	1		000a	ea	13	1f	fd	Nv-BdIzc	CLV	T0+T2	e1	e1	e1
15	000a	15	1		000a	ea	13	1f	fd	Nv-BdIzc	CLV	T0+T2	e1	e1	c2

SLO (1F)	Cycles: 7	Size: 3
	Absolute X-Indexed (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Pointer.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Pointer.H.
3.1		Inc. **PC**. Target = Pointer + X.
		Address.L = Target.L.
		Address.H = Pointer.H.
		BoundaryCrossed = Pointer.H != Target.H.
3.2	Read Address	Store as Operand.
4.1		Address.H = Target.H.
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.
6.1		**C** flag = (Operand & $80).
		Operand <<= 1.
		A |= Operand.
		N flag = (**A** & $80), **Z** flag = !**A**.
6.2	Write to Address	Write Operand.
7.1		
7.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	1f	1	unknown	0006	ea	13	1f	fd	nv-BdIzc	LDX #	T1	13	13	7c
7	0006	1f	1	unknown	0006	ea	13	1f	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	ea	13	1f	fd	nv-BdIzc	unknown	T2	26	26	26
8	0007	44	1		0007	ea	13	1f	fd	nv-BdIzc	unknown	T2	26	26	4c
9	0008	01	1		0008	ea	13	1f	fd	nv-BdIzc	unknown	T3	13	44	4c
9	0008	01	1		0008	ea	13	1f	fd	nv-BdIzc	unknown	T3	13	44	57
10	0157	e2	1		0009	ea	13	1f	fd	nv-BdIzc	unknown	T4	00	01	57
10	0157	e2	1		0009	ea	13	1f	fd	nv-BdIzc	unknown	T4	00	01	02
11	0157	e2	1		0009	ea	13	1f	fd	nv-BdIzc	unknown	T5	02	ff	02
11	0157	e2	1		0009	ea	13	1f	fd	nv-BdIzc	unknown	T5	02	ff	01
12	0157	e2	0		0009	ea	13	1f	fd	nv-BdIzc	unknown		e2	e2	01
12	0157	e2	0		0009	ea	13	1f	fd	nv-BdIzc	unknown		e2	e2	c4
13	0157	e2	0		0009	ea	13	1f	fd	Nv-BdIzC	unknown	T0	c4	c4	c4
13	0157	c4	0		0009	ea	13	1f	fd	Nv-BdIzC	unknown	T0	c4	c4	88
14	0009	b8	1	CLV	0009	ea	13	1f	fd	Nv-BdIzC	unknown	T1	ea	c4	88
14	0009	b8	1	CLV	0009	ea	13	1f	fd	Nv-BdIzC	unknown	T1	ea	c4	ee
15	000a	15	1		000a	ee	13	1f	fd	Nv-BdIzC	CLV	T0+T2	ee	ee	ee
15	000a	15	1		000a	ee	13	1f	fd	Nv-BdIzC	CLV	T0+T2	ee	ee	dc

JSR (20)	Cycles: 6	Size: 3
	Absolute (Jump)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read (**S** | $0100)	Store as Operand.
3.1		
3.2	Write to u8(**S**+0) | $0100	Write **PC**.H.
4.1		
4.2	Write to u8(**S**-1) | $0100	Write **PC**.L.
5.1		
5.2	Read **PC**	Store as Address.H.
6.1		Dec. **S** by 2. **PC** = Address.
6.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	20	1	JSR Abs	0006	ea	13	1f	fd	nv-BdIzc	LDX #	T1	13	13	7c
7	0006	20	1	JSR Abs	0006	ea	13	1f	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	ea	13	1f	fd	nv-BdIzc	JSR Abs	T2	26	26	26
8	0007	44	1		0007	ea	13	1f	fd	nv-BdIzc	JSR Abs	T2	26	26	4c
9	01fd	00	1		0008	ea	13	1f	44	nv-BdIzc	JSR Abs	T3	00	fd	4c
9	01fd	00	1		0008	ea	13	1f	44	nv-BdIzc	JSR Abs	T3	00	fd	fd
10	01fd	00	0		0008	ea	13	1f	44	nv-BdIzc	JSR Abs	T4	ff	fd	fd
10	01fd	00	0		0008	ea	13	1f	44	nv-BdIzc	JSR Abs	T4	ff	fd	fc
11	01fc	00	0		0008	ea	13	1f	44	nv-BdIzc	JSR Abs	T5	ff	fc	fc
11	01fc	08	0		0008	ea	13	1f	44	nv-BdIzc	JSR Abs	T5	ff	fc	fb
12	0008	01	1		0008	ea	13	1f	44	nv-BdIzc	JSR Abs	T0	fb	00	fb
12	0008	01	1		0008	ea	13	1f	44	nv-BdIzc	JSR Abs	T0	fb	00	fb
13	0144	ea	1	NOP	0144	ea	13	1f	fb	nv-BdIzc	JSR Abs	T1	00	01	fb
13	0144	ea	1	NOP	0144	ea	13	1f	fb	nv-BdIzc	JSR Abs	T1	00	01	01
14	0145	00	1		0145	ea	13	1f	fb	nv-BdIzc	NOP	T0+T2	01	01	01
14	0145	00	1		0145	ea	13	1f	fb	nv-BdIzc	NOP	T0+T2	01	01	02

AND (21)	Cycles: 6	Size: 2
	Indirect X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Operand.
2.1		Inc. **PC**.
2.2	Read Operand	Discard.
3.1		Pointer = u8(Operand + **X**).
3.2	Read Pointer	Store as Address.
4.1		
4.2	Read u8(Pointer + 1)	Store as Address.H.
5.1		
5.2	Read Address	Store as Operand.
6.1		**A** &= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
6.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	21	1	AND (zp,X)	0006	1f	13	1f	fd	nv-BdIzc	LDX #	T1	13	13	7c
7	0006	21	1	AND (zp,X)	0006	1f	13	1f	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T2	26	26	26
8	0007	44	1		0007	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T2	26	26	4c
9	0044	66	1		0008	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T3	13	44	4c
9	0044	66	1		0008	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T3	13	44	57
10	0057	77	1		0008	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T4	00	57	57
10	0057	77	1		0008	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T4	00	57	58
11	0058	01	1		0008	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T5	00	77	58
11	0058	01	1		0008	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T5	00	77	77
12	0177	81	1		0008	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T0	ff	ff	77
12	0177	81	1		0008	1f	13	1f	fd	nv-BdIzc	AND (zp,X)	T0	ff	ff	fe
13	0008	ea	1	NOP	0008	1f	13	1f	fd	Nv-BdIzc	AND (zp,X)	T1	1f	81	fe
13	0008	ea	1	NOP	0008	1f	13	1f	fd	Nv-BdIzc	AND (zp,X)	T1	1f	81	01
14	0009	b8	1		0009	01	13	1f	fd	nv-BdIzc	NOP	T0+T2	01	01	01
14	0009	b8	1		0009	01	13	1f	fd	nv-BdIzc	NOP	T0+T2	01	01	02

JAM (22)	Cycles: ∞	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read PC	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read PC	Discard.
2.1		Visual6502 suggests incrementing **PC**, but Single-Step Tests cannot be passed this way.
2.2	Read $FFFF	Discard.
3.1		
3.2	Read $FFFE	Discard.
4.1		
4.2	Read $FFFE	Discard.
5.1		Repeat 5.1 and 5.2 forever.
5.2	Read $FFFF	Discard.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	22	1	unknown	0006	1f	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	22	1	unknown	0006	1f	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	1f	13	22	fd	nv-BdIzc	unknown	T2	26	26	26
8	0007	44	1		0007	1f	13	22	fd	nv-BdIzc	unknown	T2	26	26	4c
9	ffff	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T3	ff	ff	4c
9	ffff	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T3	ff	ff	fe
10	fffe	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T4	ff	ff	fe
10	fffe	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T4	ff	ff	fe
11	fffe	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T5	ff	ff	fe
11	fffe	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T5	ff	ff	fe
12	ffff	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown		ff	ff	fe
12	ffff	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown		ff	ff	fe
13	ffff	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown		ff	ff	fe
13	ffff	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown		ff	ff	fe
14	ffff	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown		ff	ff	fe
14	ffff	00	1		0008	1f	13	22	fd	nv-BdIzc	unknown		ff	ff	fe

RLA (23)	Cycles: 8	Size: 2
	Indirect X (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Operand.
2.1		Inc. **PC**.
2.2	Read Operand	Discard.
3.1		Pointer = u8(Operand + **X**).
3.2	Read Pointer	Store as Address.
4.1		
4.2	Read u8(Pointer + 1)	Store as Address.H.
5.1		
5.2	Read Address	Store as Operand.
6.1		
6.2	Write to Address	Write Operand.
7.1		LowBit = **C** flag. **C** flag = (Operand & $80). Operand = (Operand << 1) | LowBit. **A** &= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
7.2	Write to Address	Write Operand.
8.1		
8.2	Read **PC**	Store as OpCode.
+X.1		* **A** applied.

* Setting of A is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	23	1	unknown	0006	1f	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	23	1	unknown	0006	1f	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	1f	13	22	fd	nv-BdIzc	unknown	T2	26	26	26
8	0007	44	1		0007	1f	13	22	fd	nv-BdIzc	unknown	T2	26	26	4c
9	0044	66	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T3	13	44	4c
9	0044	66	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T3	13	44	57
10	0057	77	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T4	00	57	57
10	0057	77	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T4	00	57	58
11	0058	01	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T5	00	77	58
11	0058	01	1		0008	1f	13	22	fd	nv-BdIzc	unknown	T5	00	77	77
12	0177	81	1		0008	1f	13	22	fd	nv-BdIzc	unknown		ff	ff	77
12	0177	81	1		0008	1f	13	22	fd	nv-BdIzc	unknown		ff	ff	fe
13	0177	81	0		0008	1f	13	22	fd	nv-BdIzc	unknown		81	81	fe
13	0177	81	0		0008	1f	13	22	fd	nv-BdIzc	unknown		81	81	02
14	0177	81	0		0008	1f	13	22	fd	nv-BdIzC	unknown	T0	02	02	02
14	0177	02	0		0008	1f	13	22	fd	nv-BdIzC	unknown	T0	02	02	04
15	0008	ea	1	NOP	0008	1f	13	22	fd	nv-BdIzC	unknown	T1	1f	02	04
15	0008	ea	1	NOP	0008	1f	13	22	fd	nv-BdIzC	unknown	T1	1f	02	02
16	0009	b8	1		0009	02	13	22	fd	nv-BdIzC	NOP	T0+T2	02	02	02
16	0009	b8	1		0009	02	13	22	fd	nv-BdIzC	NOP	T0+T2	02	02	04

BIT (24)	Cycles: 3	Size: 2
	Zero Page (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read Address	Store as Operand.
3.1		**V** flag = (Operand & $40), **N** flag = (Operand & $80), **Z** flag = !(Operand & **A**).
3.2	Read **PC**	Store as OpCode.
+X.1		* **Z** flag applied.

* Setting of Z flag is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	24	1	BIT zp	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	24	1	BIT zp	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	19	13	22	fd	nv-BdIzc	BIT zp	T2	26	26	26
8	0007	44	1		0007	19	13	22	fd	nv-BdIzc	BIT zp	T2	26	26	4c
9	0044	e6	1		0008	19	13	22	fd	nv-BdIzc	BIT zp	T0	ff	44	4c
9	0044	e6	1		0008	19	13	22	fd	nv-BdIzc	BIT zp	T0	ff	44	43
10	0008	ea	1	NOP	0008	19	13	22	fd	NV-BdIzc	BIT zp	T1	19	e6	43
10	0008	ea	1	NOP	0008	19	13	22	fd	NV-BdIzc	BIT zp	T1	19	e6	00
11	0009	b8	1		0009	19	13	22	fd	NV-BdIZc	NOP	T0+T2	00	00	00
11	0009	b8	1		0009	19	13	22	fd	NV-BdIZc	NOP	T0+T2	00	00	00

AND (25)	Cycles: 3	Size: 2
	Zero Page (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read Address	Store as Operand.
3.1		**A** &= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
3.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	25	1	AND zp	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	25	1	AND zp	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	19	13	22	fd	nv-BdIzc	AND zp	T2	26	26	26
8	0007	44	1		0007	19	13	22	fd	nv-BdIzc	AND zp	T2	26	26	4c
9	0044	e6	1		0008	19	13	22	fd	nv-BdIzc	AND zp	T0	ff	44	4c
9	0044	e6	1		0008	19	13	22	fd	nv-BdIzc	AND zp	T0	ff	44	43
10	0008	ea	1	NOP	0008	19	13	22	fd	Nv-BdIzc	AND zp	T1	19	e6	43
10	0008	ea	1	NOP	0008	19	13	22	fd	Nv-BdIzc	AND zp	T1	19	e6	00
11	0009	b8	1		0009	00	13	22	fd	nv-BdIZc	NOP	T0+T2	00	00	00
11	0009	b8	1		0009	00	13	22	fd	nv-BdIZc	NOP	T0+T2	00	00	00

ROL (26)	Cycles: 5	Size: 2
	Zero Page (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read Address	Store as Operand.
3.1		
3.2	Write to Address	Write Operand.
4.1		LowBit = **C** flag. **C** flag = (Operand & $80). Operand = (Operand << 1) | LowBit. **N** flag = (Operand & $80), **Z** flag = !Operand.
4.2	Write to Address	Write Operand.
5.1		
5.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	26	1	ROL zp	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	26	1	ROL zp	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	19	13	22	fd	nv-BdIzc	ROL zp	T2	26	26	26
8	0007	44	1		0007	19	13	22	fd	nv-BdIzc	ROL zp	T2	26	26	4c
9	0044	e6	1		0008	19	13	22	fd	nv-BdIzc	ROL zp	T3	ff	44	4c
9	0044	e6	1		0008	19	13	22	fd	nv-BdIzc	ROL zp	T3	ff	44	43
10	0044	e6	0		0008	19	13	22	fd	nv-BdIzc	ROL zp	T4	e6	e6	43
10	0044	e6	0		0008	19	13	22	fd	nv-BdIzc	ROL zp	T4	e6	e6	cc
11	0044	e6	0		0008	19	13	22	fd	Nv-BdIzC	ROL zp	T0	cc	cc	cc
11	0044	cc	0		0008	19	13	22	fd	Nv-BdIzC	ROL zp	T0	cc	cc	98
12	0008	ea	1	NOP	0008	19	13	22	fd	Nv-BdIzC	ROL zp	T1	ff	cc	98
12	0008	ea	1	NOP	0008	19	13	22	fd	Nv-BdIzC	ROL zp	T1	ff	cc	cc
13	0009	b8	1		0009	19	13	22	fd	Nv-BdIzC	NOP	T0+T2	cc	cc	cc
13	0009	b8	1		0009	19	13	22	fd	Nv-BdIzC	NOP	T0+T2	cc	cc	98

RLA (27)	Cycles: 5	Size: 2
	Zero Page (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read Address	Store as Operand.
3.1		
3.2	Write to Address	Write Operand.
4.1		LowBit = **C** flag. **C** flag = (Operand & $80). Operand = (Operand << 1) | LowBit. **A** &= Operand. **N** flag = (**A** & $80), **Z** flag = !A.
4.2	Write to Address	Write Operand.
5.1		
5.2	Read **PC**	Store as OpCode.
+X.1		* **N** flag applied.

* N flag initially set based off Operand in 4.1, then set again based off A with a delay of 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	27	1	unknown	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	27	1	unknown	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	44	1		0007	19	13	22	fd	nv-BdIzc	unknown	T2	26	26	26
8	0007	44	1		0007	19	13	22	fd	nv-BdIzc	unknown	T2	26	26	4c
9	0044	cc	1		0008	19	13	22	fd	nv-BdIzc	unknown	T3	ff	44	4c
9	0044	cc	1		0008	19	13	22	fd	nv-BdIzc	unknown	T3	ff	44	43
10	0044	cc	0		0008	19	13	22	fd	nv-BdIzc	unknown	T4	cc	cc	43
10	0044	cc	0		0008	19	13	22	fd	nv-BdIzc	unknown	T4	cc	cc	98
11	0044	cc	0		0008	19	13	22	fd	Nv-BdIzC	unknown	T0	98	98	98
11	0044	98	0		0008	19	13	22	fd	Nv-BdIzC	unknown	T0	98	98	30
12	0008	ea	1	NOP	0008	19	13	22	fd	Nv-BdIzC	unknown	T1	19	98	30
12	0008	ea	1	NOP	0008	19	13	22	fd	Nv-BdIzC	unknown	T1	19	98	18
13	0009	b8	1		0009	18	13	22	fd	nv-BdIzC	NOP	T0+T2	18	18	18
13	0009	b8	1		0009	18	13	22	fd	nv-BdIzC	NOP	T0+T2	18	18	30

PLP (28)	Cycles: 4	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Discard.
2.1		
2.2	Read (**S** | $0100)	Store as Operand.
3.1		Inc. **S** by 1.
3.2	Read (**S** | $0100)	Store as Operand.
4.1		**P** = ((Operand & ~(**X** flag)) | (**M** flag)).
4.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	28	1	PLP	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	28	1	PLP	0006	19	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	ea	1		0007	19	13	22	fd	nv-BdIzc	PLP	T2	26	26	26
8	0007	ea	1		0007	19	13	22	fd	nv-BdIzc	PLP	T2	26	26	4c
9	01fd	00	1		0007	19	13	22	fd	nv-BdIzc	PLP	T3	00	fd	4c
9	01fd	00	1		0007	19	13	22	fd	nv-BdIzc	PLP	T3	00	fd	fe
10	01fe	00	1		0007	19	13	22	fe	nv-BdIzc	PLP	T0	00	fe	fe
10	01fe	00	1		0007	19	13	22	fe	nv-BdIzc	PLP	T0	00	fe	ff
11	0007	ea	1	NOP	0007	19	13	22	fe	nv-Bdizc	PLP	T1	00	00	ff
11	0007	ea	1	NOP	0007	19	13	22	fe	nv-Bdizc	PLP	T1	00	00	01
12	0008	ea	1		0008	19	13	22	fe	nv-Bdizc	NOP	T0+T2	00	00	00
12	0008	ea	1		0008	19	13	22	fe	nv-Bdizc	NOP	T0+T2	00	01	02

AND (29)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Operand.
2.1		Inc. **PC**. A &= Operand. **N** flag = (**A** & $80), **Z** flag = !**A**.
2.2	Read **PC**	Store as OpCode.
+X.1		

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	29	1	AND #	0006	fe	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	29	1	AND #	0006	fe	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	83	1		0007	fe	13	22	fd	nv-BdIzc	AND #	T0+T2	26	26	26
8	0007	83	1		0007	fe	13	22	fd	nv-BdIzc	AND #	T0+T2	26	26	4c
9	0008	ea	1	NOP	0008	fe	13	22	fd	Nv-BdIzc	AND #	T1	fe	83	4c
9	0008	ea	1	NOP	0008	fe	13	22	fd	Nv-BdIzc	AND #	T1	fe	83	82
10	0009	b8	1		0009	82	13	22	fd	Nv-BdIzc	NOP	T0+T2	82	82	82
10	0009	b8	1		0009	82	13	22	fd	Nv-BdIzc	NOP	T0+T2	82	82	04

ROL (2A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Discard.
2.1		LowBit = **C** flag. **C** flag = (A & $80).
		A = (**A** << 1) | LowBit.
		N flag = (**A** & $80), **Z** flag = !**A**.
2.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	2a	1	ROL	0006	fe	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	2a	1	ROL	0006	fe	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	ea	1		0007	fe	13	22	fd	nv-BdIzc	ROL	T0+T2	26	26	26
8	0007	ea	1		0007	fe	13	22	fd	nv-BdIzc	ROL	T0+T2	26	26	4c
9	0007	ea	1	NOP	0007	fe	13	22	fd	nv-BdIzc	ROL	T1	fe	fe	4c
9	0007	ea	1	NOP	0007	fe	13	22	fd	nv-BdIzc	ROL	T1	fe	fe	fc
10	0008	a4	1		0008	fc	13	22	fd	Nv-BdIzC	NOP	T0+T2	fc	fc	fc
10	0008	a4	1		0008	fc	13	22	fd	Nv-BdIzC	NOP	T0+T2	fc	fc	f8

ANC (2B)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Operand.
2.1		Inc. **PC**. **A** &= Operand. **C** flag = (**A** & $80), **N** flag = (**A** & $80), **Z** flag = !**A**.
2.2	Read **PC**	Store as OpCode.
+X.1		* **A**, **C** flag, **N** flag, and **Z** flag applied.

* Setting of A/flags is delayed by 1 cycle until the start of the following instruction.

Visual6502 omits the update of A; flags are updated based off the incorrect A value.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	2b	1	unknown	0006	31	13	22	fd	nv-BdIzc	LDX #	T1	13	13	88
7	0006	2b	1	unknown	0006	31	13	22	fd	nv-BdIzc	LDX #	T1	13	13	26
8	0007	11	1		0007	31	13	22	fd	nv-BdIzc	unknown	T0+T2	26	26	26
8	0007	11	1		0007	31	13	22	fd	nv-BdIzc	unknown	T0+T2	26	26	4c
9	0008	ea	1	NOP	0008	31	13	22	fd	nv-BdIzc	unknown	T1	11	11	4c
9	0008	ea	1	NOP	0008	31	13	22	fd	nv-BdIzc	unknown	T1	31	31	31
10	0009	b8	1		0009	31	13	22	fd	nv-BdIzc	NOP	T0+T2	31	31	31
10	0009	b8	1		0009	31	13	22	fd	nv-BdIzc	NOP	T0+T2	31	31	62

ANC (ANC2, ANA, ANB)

Type: Combination of an immediate and an implied command (Sub-instructions: AND, ASL/ROL)

Opc.	Mnemonic	Function	Size	Cycles	N	V	-	B	D	I	Z	C
$0B	ANC #imm	A = A & #{imm}	2	2	o						o	o
$2B	ANC #imm	A = A & #{imm}	2	2	o						o	o

Operation: ANDs the contents of the A register with an immediate value and then moves bit 7 of A into the Carry flag.

- This opcode works basically identically to AND #imm. except that the Carry flag is set to the same state that the Negative flag is set to. (bit 7 is put into the carry, as if the ASL/ROL would have been executed)

— NoMoreSecrets-NMOS6510UnintendedOpcodes-20232412.pdf

BIT (2C)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**	Store as Address.L.
2.1		Inc. **PC**.
2.2	Read **PC**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address	Store as Operand.
4.1		**V** flag = (Operand & $40), **N** flag = (Operand & $80), **Z** flag = !(Operand & **A**).
4.2	Read **PC**	Store as OpCode.
+X.1		* **Z** flag applied.

* Setting of Z flag is delayed by 1 cycle until the start of the following instruction.

cycle	ab	db	rw	Fetch	pc	a	x	y	s	p	Execute	State	alua	alub	alu
7	0006	2c	1	BIT Abs	0006	99	f0	22	fd	Nv-BdIzc	LDX #	T1	f0	f0	88
7	0006	2c	1	BIT Abs	0006	99	f0	22	fd	Nv-BdIzc	LDX #	T1	f0	f0	e0
8	0007	11	1		0007	99	f0	22	fd	Nv-BdIzc	BIT Abs	T2	e0	e0	e0
8	0007	11	1		0007	99	f0	22	fd	Nv-BdIzc	BIT Abs	T2	e0	e0	c0
9	0008	01	1		0008	99	f0	22	fd	Nv-BdIzc	BIT Abs	T3	00	11	c0
9	0008	01	1		0008	99	f0	22	fd	Nv-BdIzc	BIT Abs	T3	00	11	11
10	0111	66	1		0009	99	f0	22	fd	Nv-BdIzc	BIT Abs	T0	ff	11	11
10	0111	66	1		0009	99	f0	22	fd	Nv-BdIzc	BIT Abs	T0	ff	11	10
11	0009	ea	1	NOP	0009	99	f0	22	fd	nV-BdIzc	BIT Abs	T1	99	66	10
11	0009	ea	1	NOP	0009	99	f0	22	fd	nV-BdIzc	BIT Abs	T1	99	66	00
12	000a	15	1		000a	99	f0	22	fd	nV-BdIZc	NOP	T0+T2	00	00	00
12	000a	15	1		000a	99	f0	22	fd	nV-BdIZc	NOP	T0+T2	00	00	00

AND (2D)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROL (2E)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		
4.2	Write to Address:**DB**	Write Operand.L.
5.1		Set **C** based off (Operand.L & $80). Perform Operand.L = (Operand.L << 1) | **C**. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (2F)	Cycles: 5	Size: 4
	Absolute Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		**Inc. PC.**
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		**Inc. PC.**
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		**Inc. PC.**
4.2	Read Address:Bank	Store as Operand.
5.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BMI (30)	Cycles: 2	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**. Decide to branch if **N** is 1.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != **PC**.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (31)	Cycles: 5	Size: 2
	Direct Indirect Indexed (d),y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (32)	Cycles: 5	Size: 2
	Direct Indirect (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer:**DB**	Store as Operand.
6.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (33)	Cycles: 7	Size: 2
	Stack Relative Indirect Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** &= Operand. Set **N** based off (**A.**L & $80) and **Z** based off **A.**L.
7.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

BIT (34)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		Set **Z** based off (**A**.L & Operand.L), set **N** based off (Operand.L & $80), and set **V** based off (Operand.L & $40).
5.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

AND (35)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROL (36)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Set **C** based off (Operand.L & $80). Perform Operand.L = (Operand.L << 1) | **C**. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (37)	Cycles: 6	Size: 2
	Direct Indirect Indexed Long [d],y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	If **D**.l is not 0, read (Address + 1), otherwise read (Address & $FF00) | ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If **D**.l is not 0, read (Address + 2), otherwise read (Address & $FF00) | ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SEC (38)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set the **C** flag to 1.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (39)	Cycles: 4	Size: 3
	Absolute, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

DEC (3A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **A**.L -= 1, set **N** based off (**A**.L & $80), set **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TSC (3B)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set **A** to (**S**.L | $0100). Set **N** based off (**A**.H & $80) and **Z** based off **A**.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BIT (3C)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **Z** based off (**A**.L & Operand.L), set **N** based off (Operand.L & $80), and set **V** based off (Operand.L & $40).
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

AND (3D)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROL (3E)	Cycles: 7	Size: 3
	Absolute, X (Read/Modiy/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**.	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Set **C** based off (Operand.L & $80). Perform Operand.L = (Operand.L << 1) | **C**. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

AND (3F)	Cycles: 5	Size: 4
	Absolute Long, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**.	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**.	Stores as Bank.
4.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** &= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**.	Store as OpCode.
+X.1		

RTI (40)	Cycles: 6	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		Inc. **S**.L by 1 and set **S**.H to 1.
3.2	Read ((**S**.L | $0100)+0)	Store as Operand.L.
4.1		Set **P** to ((Operand.L & ~$30) | (**P** & $30)).
4.2	Read ((**S**.L | $0100)+1)	Store as Operand.L.
5.1		
5.2	Read ((**S**.L | $0100)+2)	Store as Operand.H.
6.1		Inc. **S**.L by 2 and set **S**.H to 1. Set **PC** to Operand.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (41)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC.**
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**. Set Pointer to Pointer + **X** + **D**. Address.L = Pointer.L. Address.H = **D**.H.
2.2		If **D**.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer:**DB**	Store as Operand.
7.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

WDM (42)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (43)	Cycles: 4	Size: 2
	Stack Relative (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

MVP (44)	Cycles: 7	Size: 3
	Block Move (Positive)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **DB** to Operand.L.
2.2	Read **PC**:**PB**	Stores as Bank.
3.1		Inc. **PC**.
3.2	Read **X**:Bank	Store as Operand.L.
4.1		
4.2	Write **Y**:**DB**	Write Operand.L.
5.1		Perform **A** -= 1. If **X** flag is set, perform **X**.L -= 1 and **Y**.L -= 1, otherwise perform **X** -= 1 and **Y** -= 1. If **A** is not $FFFF, perform **PC** -= 3.
5.2		
6.1		
6.2		
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (45)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LSR (46)	Cycles: 5	Size: 2
	Direct (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set **C** based off (Operand.L & $01). Perform Operand.L >>= 1. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (47)	Cycles: 6	Size: 2
	Direct Indirect Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PHA (48)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2	Write to ((**S**.L | $0100)+0)	Push **A**.L onto stack.
3.1		Dec. **S**.L by 1 and set **S**.H to 1.
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (49)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LSR (4A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set **C** based off (**A**.L & $01). Perform **A**.L >>= 1. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PHK (4B)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2	Write to ((**S**.L | $0100)+0)	Push **PB** onto stack.
3.1		Dec. **S**.L by 1 and set **S**.H to 1.
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

JMP (4C)	Cycles: 3	Size: 3
	Absolute	
Cycle	R/W	Desc.
-X.2	Read **PC:PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC:PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC:PB**	Store as Address.L.
3.1		Set **PC** to Address.
3.2	Read **PC:PB**	Store as OpCode.
+X.1		

EOR (4D)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LSR (4E)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		
4.2	Write to Address:**DB**	Write Operand.L.
5.1		Set **C** based off (Operand.L & $01). Perform Operand.L >>= 1. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (4F)	Cycles: 5	Size: 4
	Absolute Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.L.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**.
4.2	Read Address:Bank	Store as Operand.
5.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BVC (50)	Cycles: 2	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**. Decide to branch if **V** is 0.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != **PC**.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (51)	Cycles: 5	Size: 2
	Direct Indirect Indexed (d),y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (52)	Cycles: 5	Size: 2
	Direct Indirect (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer:**DB**	Store as Operand.
6.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (53)	Cycles: 7	Size: 2
	Stack Relative Indirect Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

MVN (54)	Cycles: 7	Size: 3
	Block Move (Negative)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **DB** to Operand.L.
2.2	Read **PC**:**PB**	Stores as Bank.
3.1		Inc. **PC**.
3.2	Read **X**:Bank	Store as Operand.L.
4.1		
4.2	Write **Y**:**DB**	Write Operand.L.
5.1		Perform **A** -= 1. If **X** flag is set, perform **X**.L += 1 and **Y**.L += 1, otherwise perform **X** += 1 and **Y** += 1. If **A** is not $FFFF, perform **PC** -= 3.
5.2		
6.1		
6.2		
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (55)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LSR (56)	Cycles: 6	Size: 2
	Direct, X (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Set **C** based off (Operand.L & $01). Perform Operand.L >>= 1. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (57)	Cycles: 6	Size: 2
	Direct Indirect Indexed Long [d],y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	If **D**.l is not 0, read (Address + 1), otherwise read (Address & $FF00) | ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If **D**.l is not 0, read (Address + 2), otherwise read (Address & $FF00) | ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CLI (58)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set the **I** flag to 0.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (59)	Cycles: 4	Size: 3
	Absolute, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PHY (5A)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2	Write to ((**S**.L | $0100)+0)	Push **Y**.L onto stack.
3.1		Dec. **S**.L by 1 and set **S**.H to 1.
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TCD (5B)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set **D** to **A**. Set **N** based off (**D**.H & $80) and **Z** based off **D**.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

JML (5C)	Cycles: 4	Size: 4
	Absolute Long	
Cycle	R/W	Desc.
-X.2	Read **PC:PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC:PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC:PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read **PC:PB**	Stores as Bank.
4.1		Copy Address to **PC**. Copy Bank to **PB**.
4.2	Read **PC:PB**	Store as OpCode.
+X.1		

EOR (5D)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**.	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**.	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer.
		Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles.
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LSR (5E)	Cycles: 7	Size: 3
	Absolute, X (Read/Modiy/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Set **C** based off (Operand.L & $01). Perform Operand.L >>= 1. Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

EOR (5F)	Cycles: 5	Size: 4
	Absolute Long, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand.
5.1		Perform **A** ^= Operand. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

RTS (60)	Cycles: 6	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		Inc. **S**.L by 1 and set **S**.H to 1.
3.2	Read ((**S**.L | $0100)+0)	Store as Operand.L.
4.1		
4.2	Read ((**S**.L | $0100)+1)	Store as Operand.H.
5.1		
5.2		
6.1		Inc. **S**.L by 1 and set **S**.H to 1. Set **PC** to (Operand + 1).
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (61)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Pointer to Pointer + **X** + **D**. Address.L = Pointer.L. Address.H = **D**.H.
2.2		If **D**.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer:**DB**	Store as Operand.

7.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PER (62)	Cycles: 6	Size: 3
	Program Counter Relative Long	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Operand.H.
3.1		Inc. **PC**. Set Operand to Operand + **PC**.
3.2		
4.1		
4.2	Write to ((**S**.L | $0100)+0)	Push Operand.H onto stack.
5.1		
5.2	Write to ((**S**.L | $0100)-1)	Push Operand.L onto stack.
6.1		Dec. **S**.L by 2 and set **S**.H to 1.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (63)	Cycles: 4	Size: 2
	Stack Relative (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.

4.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STZ (64)	Cycles: 3	Size: 2
	Direct (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Write to Address	Write 0.
4.1		
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (65)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.

4.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROR (66)	Cycles: 5	Size: 2
	Direct (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Set **C** based off (Operand.L & $01). Perform Operand.L = (Operand.L >> 1) | (**C** << 7). Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (67)	Cycles: 6	Size: 2
	Direct Indirect Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.

7.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PLA (68)	Cycles: 4	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		Inc. **S**.L by 1 and set **S**.H to 1.
3.2	Read u8(**S**.L+0) | $0100	Store as Operand.L.
4.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (69)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.

2.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROR (6A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set **C** based off (**A**.L & $01). Perform **A**.L = (**A**.L >> 1) | (**C** << 7). Set **N** based off (**A**.L & $80) and **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

RTL (6B)	Cycles: 6	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		
3.2	Read u8(**S**.L+1) | $0100	Store as Operand.L.
4.1		
4.2	Read u8(**S**.L+2) | $0100	Store as Operand.H.
5.1		
5.2	Read ((**S**.L | $0100)+3)	Store as **PB**.
6.1		Inc. **S**.L by 3 and set **S**.H to 1. Set **PC** to (Operand + 1). Set **PB** to Bank.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

JMP (6C)	Cycles: 5	Size: 3
	Absolute Indirect (Jump)	
Cycle	R/W	Desc.
-X.2	Read **PC:PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC:PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC:PB**	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read Pointer	Store as Address.
4.1		
4.2	Read Pointer + 1	Store as Address.H.
5.1		Set **PC** to Address.
5.2	Read **PC:PB**	Store as OpCode.
+X.1		

ADC (6D)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.

4.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROR (6E)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		
4.2	Write to Address:**DB**	Write Operand.L.
5.1		Set **C** based off (Operand.L & $01). Perform Operand.L = (Operand.L >> 1) | (**C** << 7). Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (6F)	Cycles: 5	Size: 4
	Absolute Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**.
4.2	Read Address:Bank	Store as Operand.

5.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BVS (70)	Cycles: 2	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**. Decide to branch if **V** is 1.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != **PC**.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (71)	Cycles: 5	Size: 2
	Direct Indirect Indexed (d),y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.

7.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (72)	Cycles: 5	Size: 2
	Direct Indirect (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer:**DB**	Store as Operand.

6.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (73)	Cycles: 7	Size: 2
	Stack Relative Indirect Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STZ (74)	Cycles: 4	Size: 2
	Direct, X (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Write to Address	Write 0.
5.1		
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (75)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.

5.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROR (76)	Cycles: 6	Size: 2
	Direct, X (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Set **C** based off (Operand.L & $01). Perform Operand.L = (Operand.L >> 1) | (**C** << 7). Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (77)	Cycles: 6	Size: 2
	Direct Indirect Indexed Long [d],y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	If **D**.l is not 0, read (Address + 1), otherwise read (Address & $FF00) | ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If **D**.l is not 0, read (Address + 2), otherwise read (Address & $FF00) | ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.

7.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SEI (78)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set the **I** flag to 1.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (79)	Cycles: 4	Size: 3
	Absolute, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer.
		Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		If Orig.H == Pointer.H, skip 2 half-cycles.
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PLY (7A)	Cycles: 4	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		Inc. **S**.L by 1 and set **S**.H to 1.
3.2	Read u8(**S**.L+0) | $0100	Store as Operand.L.
4.1	Set **Y**.L to Operand.L. Set **N** based off (**Y**.L & $80). Set **Z** based off **Y**.L.	
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TDC (7B)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set **A** to **D**. Set **N** based off (**A**.H & $80) and **Z** based off **A**.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

JMP (7C)	Cycles: 6	Size: 3
	Absolute Indexed Indirect (a,x) (Jump	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Perform Pointer += **X**. Bank = **PB**.
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Address.L.
5.1		
5.2	Read Pointer + 1:Bank	Store as Address.H.
6.1		Set **PC** to Address.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (7D)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. PC. Perform Orig = Pointer.
		Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		If Orig.H == Pointer.H, skip 2 half-cycles.
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ROR (7E)	Cycles: 7	Size: 3
	Absolute, X (Read/Modiy/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Set **C** based off (Operand.L & $01). Perform Operand.L = (Operand.L >> 1) | (**C** << 7). Set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

ADC (7F)	Cycles: 5	Size: 4
	Absolute Long, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand.
5.1		If **D** flag == 0:
		Result = ui16(**A**.L) + ui16(Operand) + **C**.
		V flag = (~(ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result > $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = ui16(**A**.L & 0x0F) + ui16(Operand & 0x0F) + **C**;
		if (Lo > 9) Lo += 6.
		CarryToHi = Lo > $0F ? 1 : 0.
		HiSum = ui16(**A**.L >> 4) + ui16(Operand >> 4) + CarryToHi.
		V flag = (~((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum > 9) HiSum += 6.
		C flag = HiSum > $0F.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BRA (80)	Cycles: 3	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**. Decide to branch always.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != **PC**.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (81)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Pointer to Pointer + X + **D**.
		Address.L = Pointer.L.
		Address.H = **D**.H.
2.2		If **D**.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		Set Operand.L to **A**.L.
6.2	Write to Pointer:**DB**	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BRL (82)	Cycles: 4	Size: 3
	Program Counter Relative Long	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Operand.H.
3.1		Inc. **PC**.
3.2		
4.1		Perform **PC** += Operand.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (83)	Cycles: 4	Size: 2
	Stack Relative (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		Set Operand to **A**.
3.2	Write to Address	Write Operand.L.
4.1		
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STY (84)	Cycles: 3	Size: 2
	Direct (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Write to Address	Write **Y**.L.
4.1		
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (85)	Cycles: 3	Size: 2
	Direct (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Write to Address	Write **A**.L.
4.1		
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STX (86)	Cycles: 3	Size: 2
	Direct (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Write to Address	Write **X**.L.
4.1		
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (87)	Cycles: 6	Size: 2
	Direct Indirect Long (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		Set Operand to **A**.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

DEY (88)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **Y**.L -= 1, set **N** based off (**Y**.L & $80), set **Z** based off **Y**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BIT (89)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **Z** based off (**A**.L & Operand.L).
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TXA (8A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **A**.L = **X**.L, set **N** based off (**A**.L & $80), set **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PHB (8B)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2	Write to ((**S**.L | $0100)+0)	Push **DB** onto stack.
3.1		Dec. **S**.L by 1 and set **S**.H to 1.
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STY (8C)	Cycles: 4	Size: 3
	Absolute (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc **PC**. Set Operand to **Y**.
3.2	Write to Address:**DB**	Write Operand.L.
4.1		
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (8D)	Cycles: 4	Size: 3
	Absolute (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC:PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC:PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC:PB**	Store as Address.H.
3.1		Inc **PC**. Set Operand to **A**.
3.2	Write to Address:**DB**	Write Operand.L.
4.1		
4.2	Read **PC:PB**	Store as OpCode.
+X.1		

STX (8E)	Cycles: 4	Size: 3
	Absolute (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC:PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC:PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC:PB**	Store as Address.H.
3.1		Inc **PC**. Set Operand to **X**.
3.2	Write to Address:**DB**	Write Operand.L.
4.1		
4.2	Read **PC:PB**	Store as OpCode.
+X.1		

STA (8F)	Cycles: 5	Size: 4
	Absolute Long (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC:PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC:PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC:PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read **PC:PB**	Stores as Bank.
4.1		Inc **PC**. Set Operand to **A**.
4.2	Write to Address:Bank	Write Operand.L.
5.1		
5.2	Read **PC:PB**	Store as OpCode.
+X.1		

BCC (90)	Cycles: 2	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**. Decide to branch if **C** is 0.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != **PC**.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (91)	Cycles: 6	Size: 2
	Direct Indirect Indexed (d),y (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
5.2		
6.1		Set Operand to **A**.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (92)	Cycles: 5	Size: 2
	Direct Indirect (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Set Operand to **A**.
5.2	Write to Pointer:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (93)	Cycles: 7	Size: 2
	Stack Relative Indirect Indexed (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
5.2		
6.1		Set Operand to **A**.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STY (94)	Cycles: 4	Size: 2
	Direct, X (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Write to Address	Write **Y**.L.
5.1		
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (95)	Cycles: 4	Size: 2
	Direct, X (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Write to Address	Write **A**.L.
5.1		
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STX (96)	Cycles: 4	Size: 2
	Direct, Y (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **Y**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **Y** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Write to Address	Write **X**.L.
5.1		
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (97)	Cycles: 6	Size: 2
	Direct Indirect Indexed Long [d],y (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	If **D**.l is not 0, read (Address + 1), otherwise read (Address & $FF00) | ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If **D**.l is not 0, read (Address + 2), otherwise read (Address & $FF00) | ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Write to Pointer:Bank	Write **A**.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TYA (98)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **A**.L = **Y**.L, set **N** based off (**A**.L & $80), set **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (99)	Cycles: 5	Size: 3
	Absolute, Y (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		Set Operand to **A**.
4.2	Write to Pointer:Bank	Write Operand.L.
5.1		
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TXS (9A)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set **S**.L to **X**.L and **S**.H to 1
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TXY (9B)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **Y**.L = **X**.L, set **N** based off (**Y**.L & $80), set **Z** based off **Y**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STZ (9C)	Cycles: 4	Size: 3
	Absolute (Write)	
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc **PC**. Set Operand to 0.
3.2	Write to Address:**DB**	Write Operand.L.
4.1		
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (9D)	Cycles: 5	Size: 3
	Absolute, X (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		Set Operand to **A**.
4.2	Write to Pointer:Bank	Write Operand.L.
5.1		
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STZ (9E)	Cycles: 5	Size: 3
	Absolute, X (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		Set Operand to 0.
4.2	Write to Pointer:Bank	Write Operand.L.
5.1		
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STA (9F)	Cycles: 5	Size: 4
	Absolute Long, X (Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank += (Tmp >> 16).
4.2	Write to Pointer:Bank	Write **A**.L.
5.1		
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDY (A0)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **Y**.L to Operand.L. Set **N** based off (**Y**.L & $80). Set **Z** based off **Y**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (A1)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Pointer to Pointer + **X** + **D**.
		Address.L = Pointer.L.
		Address.H = **D**.H.
2.2		If **D**.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer:**DB**	Store as Operand.
7.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDX (A2)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **X**.L to Operand.L. Set **N** based off (**X**.L & $80). Set **Z** based off **X**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (A3)	Cycles: 4	Size: 2
	Stack Relative (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDY (A4)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Set **Y**.L to Operand.L. Set **N** based off (**Y**.L & $80). Set **Z** based off **Y**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (A5)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDX (A6)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Set **X**.L to Operand.L. Set **N** based off (**X**.L & $80). Set **Z** based off **X**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (A7)	Cycles: 6	Size: 2
	Direct Indirect Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TAY (A8)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **Y**.L = **A**.L, set **N** based off (**Y**.L & $80), set **Z** based off **Y**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (A9)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TAX (AA)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **X**.L = **A**.L, set **N** based off (**X**.L & $80), set **Z** based off **X**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PLB (AB)	Cycles: 4	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		Inc. **S**.L by 1 and set **S**.H to 1.
3.2	Read ((**S**.L | $0100)+0)	Store as Bank.
4.1		Set **DB** to Bank, set **N** based off (**DB** & $80), and set **Z** based off **DB**.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDY (AC)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Set **Y**.L to Operand.L. Set **N** based off (**Y**.L & $80). Set **Z** based off **Y**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (AD)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDX (AE)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Set **X**.L to Operand.L. Set **N** based off (**X**.L & $80). Set **Z** based off **X**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (AF)	Cycles: 5	Size: 4
	Absolute Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**.
4.2	Read Address:Bank	Store as Operand.
5.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BCS (B0)	Cycles: 2	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. **PC**. Decide to branch if **C** is 1.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != **PC**.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (B1)	Cycles: 5	Size: 2
	Direct Indirect Indexed (d),y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (B2)	Cycles: 5	Size: 2
	Direct Indirect (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer:**DB**	Store as Operand.
6.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (B3)	Cycles: 7	Size: 2
	Stack Relative Indirect Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDY (B4)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		Set **Y**.L to Operand.L. Set **N** based off (**Y**.L & $80). Set **Z** based off **Y**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (B5)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDX (B6)	Cycles: 4	Size: 2
	Direct, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **Y**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **Y** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		Set **X**.L to Operand.L. Set **N** based off (**X**.L & $80). Set **Z** based off **X**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (B7)	Cycles: 6	Size: 2
	Direct Indirect Indexed Long [d],y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	If **D**.l is not 0, read (Address + 1), otherwise read (Address & $FF00) | ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If **D**.l is not 0, read (Address + 2), otherwise read (Address & $FF00) | ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CLV (B8)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set the **V** flag to 0.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (B9)	Cycles: 4	Size: 3
	Absolute, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TSX (BA)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **X**.L = **S**.L, set **N** based off (**X**.L & $80), set **Z** based off **X**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

TYX (BB)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **X**.L = **Y**.L, set **N** based off (**X**.L & $80), set **Z** based off **X**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDY (BC)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **Y**.L to Operand.L. Set **N** based off (**Y**.L & $80). Set **Z** based off **Y**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (BD)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDX (BE)	Cycles: 4	Size: 3
	Absolute, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **X**.L to Operand.L. Set **N** based off (**X**.L & $80). Set **Z** based off **X**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

LDA (BF)	Cycles: 5	Size: 4
	Absolute Long, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **A**.L to Operand.L. Set **N** based off (**A**.L & $80). Set **Z** based off **A**.L.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CPY (C0)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **C** flag based off (**Y**.L >= Operand.L), set **Z** flag based off (**Y**.L == Operand.L), and set **N** flag based off ((**Y**.L - Operand.L) & $80) != 0.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (C1)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read PC:PB	Store as Pointer.
2.1		Inc. **PC**. Set Pointer to Pointer + **X** + **D**.
		Address.L = Pointer.L.
		Address.H = **D**.H.
2.2		If **D**.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer:**DB**	Store as Operand.
7.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

REP (C2)	Cycles: 3	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		**P** &= ~(Operand.L & ~(**X** flag | **M** flag)).
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (C3)	Cycles: 4	Size: 2
	Stack Relative (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.
4.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CPY (C4)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Set **C** flag based off (**Y**.L >= Operand.L), set **Z** flag based off (**Y**.L == Operand.L), and set **N** flag based off ((**Y**.L - Operand.L) & $80) != 0.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (C5)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Set **C** flag based off (**A**.L >= Operand.L), set Z flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

DEC (C6)	Cycles: 5	Size: 2
	Direct (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Perform Operand.L -= 1, set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (C7)	Cycles: 6	Size: 2
	Direct Indirect Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

INY (C8)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **Y**.L += 1, set **N** based off (**Y**.L & $80), set **Z** based off **Y**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (C9)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

DEX (CA)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **X**.L -= 1, set **N** based off (**X**.L & $80), set **Z** based off **X**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

WAI (CB)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		
2.2		
3.1		
3.2		Enter waiting state. Repeat this cycle forever.

CPY (CC)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Set **C** flag based off (**Y**.L >= Operand.L), set **Z** flag based off (**Y**.L == Operand.L), and set **N** flag based off ((**Y**.L - Operand.L) & $80) != 0.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (CD)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

DEC (CE)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		
4.2	Write to Address:**DB**	Write Operand.L.
5.1		Perform Operand.L -= 1, set **N** based off (Operand.L & $80) and **Z** based off Operand.L.
5.2	Write to Address:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (CF)	Cycles: 5	Size: 4
	Absolute Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**.
4.2	Read Address:Bank	Store as Operand.
5.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BNE (D0)	Cycles: 2	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. **PC**. Decide to branch if **Z** is 0.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != **PC**.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (D1)	Cycles: 5	Size: 2
	Direct Indirect Indexed (d),y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. PC if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (D2)	Cycles: 5	Size: 2
	Direct Indirect (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer:**DB**	Store as Operand.
6.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (D3)	Cycles: 7	Size: 2
	Stack Relative Indirect Indexed (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PEI (D4)	Cycles: 6	Size: 2
	Direct Page Indirect	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	If **D**.L is 0, Read (((Address.L + 1) & $FF) | (Address.H << 8)), otherwise read Address + 1	Store as Bank.
5.1		
5.2	Write to ((**S**.L | $0100)+0)	Push Operand.H onto stack.
6.1		
6.2	Write to ((**S**.L | $0100)-1)	Push Operand.L onto stack.
7.1		Dec. **S**.L by 2 and set **S**.H to 1.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (D5)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

DEC (D6)	Cycles: 6	Size: 2
	Direct, X (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Perform Operand.L -= 1, set **N** based off (Operand.L & $80), and **Z** based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (D7)	Cycles: 6	Size: 2
	Direct Indirect Indexed Long [d],y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	If **D**.l is not 0, read (Address + 1), otherwise read (Address & $FF00) | ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If **D**.l is not 0, read (Address + 2), otherwise read (Address & $FF00) | ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.
7.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CLD (D8)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set the **D** flag to 0.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (D9)	Cycles: 4	Size: 3
	Absolute, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PHX (DA)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2	Write to ((**S**.L | $0100)+0)	Push **X**.L onto stack.
3.1		Dec. **S**.L by 1 and set **S**.H to 1.
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

STP (DB)	Cycles: 4	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		
3.2		
4.1		Stop processor.

JML (DC)	Cycles: 6	Size: 3
	Absolute Indirect Long (Jump)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read Pointer	Store as Operand.
4.1		
4.2	Read Pointer + 1	Store as Operand.H.
5.1		
5.2	If **D**.L is 0, Read (((Pointer.L + 2) & $FF) | (Pointer.H << 8)), otherwise read Pointer + 2	Store as Bank.
6.1		Set **PC** to Operand. Set **PB** to Bank.
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (DD)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

DEC (DE)	Cycles: 7	Size: 3
	Absolute, X (Read/Modiy/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Perform Operand.L -= 1, set **N** based off (Operand.L & $80), and **Z** based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CMP (DF)	Cycles: 5	Size: 4
	Absolute Long, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand.
5.1		Set **C** flag based off (**A**.L >= Operand.L), set **Z** flag based off (**A**.L == Operand.L), and set **N** flag based off ((**A**.L - Operand.L) & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CPX (E0)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**. Set **C** flag based off (**X**.L >= Operand.L), set **Z** flag based off (**X**.L == Operand.L), and set **N** flag based off ((**X**.L - Operand.L) & $80) != 0.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (E1)	Cycles: 6	Size: 2
	Direct Indexed Indirect (d,x) (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Pointer to Pointer + **X** + **D**. Address.L = Pointer.L. Address.H = **D**.H.
2.2		If **D**.L is 0, skip the next cycle.
3.1		Address.H = Pointer.H (fixes high byte of address).
3.2		
4.1		
4.2	Read Address	Store as Pointer.
5.1		
5.2	Read Address + 1	Store as Pointer.H.
6.1		
6.2	Read Pointer:**DB**	Store as Operand.

7.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SEP (E2)	Cycles: 3	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		
2.2		
3.1		Perform **P** |= (Operand.L & ~(**X** flag | **M** flag)).
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (E3)	Cycles: 4	Size: 2
	Stack Relative (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Operand.

4.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CPX (E4)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		Set **C** flag based off (**X**.L >= Operand.L), set **Z** flag based off (**X**.L == Operand.L), and set **N** flag based off ((**X**.L - Operand.L) & $80) != 0.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (E5)	Cycles: 3	Size: 2
	Direct (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.

4.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

INC (E6)	Cycles: 5	Size: 2
	Direct (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Operand.
4.1		
4.2	Write to Address	Write Operand.L.
5.1		Perform Operand.L += 1, set **N** based off (Operand.L & $80), set **Z** based off Operand.L.
5.2	Write to Address	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (E7)	Cycles: 6	Size: 2
	Direct Indirect Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Address + 2	Store as Bank.
6.1		
6.2	Read Pointer:Bank	Store as Operand.

7.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

INX (E8)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Perform **X**.L += 1, set **N** based off (**X**.L & $80), set **Z** based off **X**.L.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (E9)	Cycles: 2	Size: 2
	Immediate	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. PC.
1.2	Read **PC**:**PB**	Store as Operand.

2.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

NOP (EA)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

XBA (EB)	Cycles: 3	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		Swap **A**.L and **A**.H, set **N** based off (**A**.L & $80), and set **Z** based off **A**.L.
3.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

CPX (EC)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		Set **C** flag based off (**X**.L >= Operand.L), set **Z** flag based off (**X**.L == Operand.L), and set **N** flag based off ((**X**.L - Operand.L) & $80) != 0.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (ED)	Cycles: 4	Size: 3
	Absolute (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.

4.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

INC (EE)	Cycles: 6	Size: 3
	Absolute (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read PC:PB	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read Address:**DB**	Store as Operand.
4.1		
4.2	Write to Address:**DB**	Write Operand.L.
5.1		Perform Operand.L += 1, set **N** based off (Operand.L & $80), set **Z** based off Operand.L.
5.2	Write to Address:**DB**	Write Operand.L.
6.1		
6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (EF)	Cycles: 5	Size: 4
	Absolute Long (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Address.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Address.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**.
4.2	Read Address:Bank	Store as Operand.

5.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

BEQ (F0)	Cycles: 2	Size: 2
	Relative	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**. Decide to branch if **Z** is 1.
1.2	Read **PC**:**PB**	Store as Operand. Address = i16(i8(Operand.L)) + **PC**, BoundaryCrossed = Address.H != **PC**.H. If not branching or not BoundaryCrossed, poll interrupts.
2.1		Inc. **PC**. If not branching, end (next half-cycle is 4.2).
2.2		
3.1		Set **PC**.L to Address.L. If not BoundaryCrossed, end (next half-cycle is 4.2).
3.2		Poll interrupts.
4.1		Set **PC**.H to Address.H
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (F1)	Cycles: 5	Size: 2
	Direct Indirect Indexed (d),y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.

7.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (F2)	Cycles: 5	Size: 2
	Direct Indirect (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If D.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		
5.2	Read Pointer:**DB**	Store as Operand.

6.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

6.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (F3)	Cycles: 7	Size: 2
	Stack Relative Indirect Indexed	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**. Set Address to Pointer + (**S**.L | $0100).
2.2		
3.1		
3.2	Read Address	Store as Pointer.
4.1		
4.2	Read Address + 1	Store as Pointer.H.
5.1		Perform Tmp = ui32(Pointer + **Y**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Operand.
7.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PEA (F4)	Cycles: 5	Size: 3
	Absolute (Push)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Operand.H.
3.1		Inc. **PC**.
3.2	Write to ((**S**.L | $0100)+0)	Push Operand.H onto stack.
4.1		
4.2	Write to ((**S**.L | $0100)-1)	Push Operand.L onto stack.
5.1		Dec. **S**.L by 2 and set **S**.H to 1.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (F5)	Cycles: 4	Size: 2
	Direct, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.

5.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

INC (F6)	Cycles: 6	Size: 2
	Direct, X (Read/Modify/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand.
2.1		Inc. **PC**.
2.2		
3.1		If **D**.L is 0, Address.L = Operand + **X**.L + **D**.L, Address.H = **D**.H, otherwise Address = Operand + **X** + **D**. If **D**.L is 0, skip 2 half-cycles.
3.2		
4.1		
4.2	Read Address	Store as Operand.
5.1		
5.2	Write to Address	Write Operand.L.
6.1		Perform Operand.L += 1, set **N** based off (Operand.L & $80), set **Z** based off Operand.L.
6.2	Write to Address	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (F7)	Cycles: 6	Size: 2
	Direct Indirect Indexed Long [d],y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Operand. If **D**.L is 0, skip the next cycle.
2.1		Inc. **PC**.
2.2		
3.1		Inc. **PC** if previous cycle was skipped. Set Address to Operand + **D**.
3.2	Read Address	Store as Pointer.
4.1		
4.2	If **D**.l is not 0, read (Address + 1), otherwise read (Address & $FF00) | ui8(Address.L + 1)	Store as Pointer.H.
5.1		
5.2	If **D**.l is not 0, read (Address + 2), otherwise read (Address & $FF00) | ui8(Address.L + 2)	Store as Bank.
6.1		Perform Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). Bank += (Tmp >> 16).
6.2	Read Pointer:Bank	Store as Operand.

7.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SED (F8)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Set the **D** flag to 1.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (F9)	Cycles: 4	Size: 3
	Absolute, Y (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. PC. Perform Orig = Pointer. Tmp = ui32(Pointer + **Y**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.

5.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

PLX (FA)	Cycles: 4	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2		
2.1		
2.2		
3.1		Inc. **S**.L by 1 and set **S**.H to 1.
3.2	Read u8(**S**.L+0) | $0100	Store as Operand.L.
4.1		Set **X**.L to Operand.L, set **N** based off (**X**.L & $80), and set **Z** based off **X**.L.
4.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

XCE (FB)	Cycles: 2	Size: 1
	Implied	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Discard.
2.1		Exchanges the Carry and Emulation flags. If Emulation mode is entered, **M** and **X** flags are set to 1, **S**.H is set to 1, and **X**.H and **Y**.H are cleared to 0.
2.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

JSR (FC)	Cycles: 8	Size: 3
	Absolute Indexed Indirect (a,x) (Jump)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Write to u8(**S**.L+0) | $0100	Push **PC**.H onto stack.
3.1		
3.2	Write to u8(**S**.L-1) | $0100	Push **PC**.L onto stack.
4.1		Dec. **S**.L by 2 and set **S**.H to 1.
4.2	Read **PC**:**PB**	Store as Pointer.H.
5.1		Perform Pointer += **X**.
		Bank = **PB**.
5.2		
6.1		
6.2	Read Pointer:Bank	Store as Address.L.
7.1		
7.2	Read Pointer + 1:Bank	Store as Address.H.
8.1		Set **PC** to Address.
8.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (FD)	Cycles: 4	Size: 3
	Absolute, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Orig = Pointer. Tmp = ui32(Pointer + **X**). Pointer = ui16(Tmp). If Orig.H == Pointer.H, skip 2 half-cycles. Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.

5.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.

5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

INC (FE)	Cycles: 7	Size: 3
	Absolute, X (Read/Modiy/Write)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank = **DB** + (Tmp >> 16).
3.2		
4.1		
4.2	Read Pointer:Bank	Store as Operand.
5.1		
5.2	Write to Pointer:Bank	Write Operand.L.
6.1		Perform Operand.L += 1, set **N** based off (Operand.L & $80), set **Z** based off Operand.L.
6.2	Write to Pointer:Bank	Write Operand.L.
7.1		
7.2	Read **PC**:**PB**	Store as OpCode.
+X.1		

SBC (FF)	Cycles: 5	Size: 4
	Absolute Long, X (Read)	
Cycle	R/W	Desc.
-X.2	Read **PC**:**PB**	Store as OpCode.
1.1		Inc. **PC**.
1.2	Read **PC**:**PB**	Store as Pointer.
2.1		Inc. **PC**.
2.2	Read **PC**:**PB**	Store as Pointer.H.
3.1		Inc. **PC**.
3.2	Read **PC**:**PB**	Stores as Bank.
4.1		Inc. **PC**. Perform Tmp = ui32(Pointer + **X**).
		Pointer = ui16(Tmp).
		Bank += (Tmp >> 16).
4.2	Read Pointer:Bank	Store as Operand.
5.1		If **D** flag == 0:
		Result = ui16(**A**.L) - ui16(Operand) - (1 - **C**).
		V flag = ((ui16(**A**.L) ^ ui16(Operand)) & (ui16(**A**.L) ^ ui8(Result)) & $80) != 0.
		C flag = Result <= $FF.
		Result = ui8(Result). **Z** flag = Result == $00. **N** flag = (Result & $80) != 0.
		If **D** flag == 1:
		Lo = int16(**A**.L & 0x0F) - int16(Operand & 0x0F) - (1 - **C**);
		if (Lo < 0) Lo -= 6.
		BorrowToHi = Lo < 0 ? 1 : 0.
		HiSum = int16(**A**.L >> 4) - int16(Operand >> 4) - BorrowToHi.
		V flag = (((**A**.L >> 4) ^ (Operand >> 4)) & ((**A**.L >> 4) ^ HiSum) & $08) != 0.
		if (HiSum < 0) HiSum -= 6.
		C flag = HiSum >= 0.
		Result = ui8(((HiSum & 0x0F) << 4) | (Lo & $0F)).
		Z flag = Result == $00. **N** flag = (Result & $80) != 0.
5.2	Read **PC**:**PB**	Store as OpCode.
+X.1		